

HelpAI

Shipping AI-powered iOS apps with Expo, React Native, `llama.cpp` and Metal

Expo SDK 54 field manual · Version 1

A practical record of problems that cost real development time, the evidence that identified them, and the fixes that worked in shipped or device-tested personal iPhone apps.

Eoin Tolster · Forward Deployed AI Engineer · Australia

[YouTube](#) · [LinkedIn](#) · theeoin.com

Primary case studies: Localresearch, DotPhysics, LongHorizon, Melody Mind, My Pet Slime, Imperial Dreams, and anonymised personal iPhone projects

Edition reviewed: 13 August 2026

Version and store-status note

This edition is deliberately anchored to **Expo SDK 54**, React Native 0.81 and React 19.1 because that is the proven baseline across the source projects. As of this edition, Expo states that SDK 54 EAS builds use Xcode 26 by default and meet Apple's iOS 26 SDK upload requirement. Apple accepts the resulting binary and toolchain output—not an Expo version number—so recheck current Apple and Expo requirements before every release. Do not interpret this guide as a permanent promise that SDK 54 will remain submission-compatible.

How to interpret the claims

Platform requirement means a current Apple, Expo or npm rule. **Repo baseline** means a version or convention deliberately held across these apps. **Observed fix** means it solved a specific failure in one of the author's personal projects; it is a diagnostic lead, not a universal framework rule. Source, export, simulator, TestFlight and physical-device evidence are kept separate throughout.

Disclosure note

Named case studies are the author's personal projects. Credentials, private account identifiers and bundle identifiers for non-public projects are excluded. If this guide is adapted for employer or client work, anonymise incident names, logs and architecture details again before publication.

Who This Is For

AI engineers putting a model on a phone for the first time. Part 3 covers the route from `llama.cpp` to a working offline app, including the native build details that high-level setup guides often omit.

iOS and React Native developers using Expo who have hit a wall that Expo Go cannot explain. Parts 3 to 8 focus on native boundaries, real-time rendering, release-only failures and build-pipeline diagnosis.

Hobbyists shipping a first app. Start with Part 1, read the short checklist in Part 2, then follow Part 9 for the store workflow.

How To Read It

Most entries follow the same shape: **Problem, Root cause, Solution, Rule**. The Rule is the durable part; the surrounding evidence helps determine whether the same diagnosis applies to your app. An **Observed in** label records provenance, not a claim that the named app is the only place the pattern applies.

Where a lesson came out of a specific app, that app is named under the heading and links to its entry in [Appendix A](#).

Contents

::: {.contents-grid}

Part 1 — [The Standard Setup](#) — The shape every one of these apps starts from, and the pieces Apple requires before a build can ship.

- [A Standard Expo SDK 54 App](#)
- [What The App Store Requires](#)
- [When An App Strays From The Default](#)

Part 2 — [Start Here](#) — The short version. If you read nothing else, read this part before your next build.

- [Current Expo Baseline](#)
- [Key Takeaways](#)
- [Pre-Build Checklist](#)
- [Starting a New App for App Store](#)

Part 3 — [Running LLMs On The Device](#) — Getting a real GGUF model running inside an Expo app, and keeping it offline afterwards.

- [Local LLM Setup With Expo And llama.cpp](#)

Part 4 — [Native Modules: Swift, Objective-C++ And Metal](#) — Where Expo stops and native code starts, and how to make the two meet without fighting the build.

- [Native Expo Module Development \(Finger Shoot\)](#)
- [Native Metal Particle Engine — iOS Learnings](#)
- [LongHorizon Swift/Metal Battle Renderer - Additional Learnings](#)

Part 5 — [Real-Time Games: Skia, Reanimated And Simulation](#) — Patterns for anything that runs a loop every frame instead of re-rendering on state change.

- [Game Development Patterns \(Three Finger Shoot\)](#)
- [Large Skia Campaign Case Study — August 2026](#)
- [Reanimated, Simulation And Layout Case Study — July-August 2026](#)

Part 6 — [Production Crashes And Their Causes](#) — Failures that only appear in a release build, on a real device, or on a clean install.

- [Critical Production Issues](#)

Part 7 — [Assets, Dependencies, Code Structure And Routing](#) — The everyday traps that cost hours and have unremarkable fixes.

- [Asset Issues](#)
- [Dependency Management](#)
- [Code Patterns That Work](#)
- [Expo Router Issues](#)

Part 8 — [Warnings: Classify Before Ignoring](#) — Distinguishing informational messages, migration debt and actionable route or build failures.

- [Warning Triage](#)
- [Common Confusing Messages \(NOT Errors\)](#)

Part 9 — [Building, Submitting And Shipping](#) — Getting from a working local app to a build that App Store Connect will actually accept.

- [Deploy Workflow](#)
- [EAS Submit Config](#)

Appendices

- [Appendix A — Public Apps Referenced](#)
- [Appendix B — Pre-Flight Checklist](#)
- [Appendix C — Index Of Every Lesson](#)
- [Appendix D — Versioned References](#)

...

Part 1 — The Standard Setup

The shape every one of these apps starts from, and the pieces Apple requires before a build can ship.

A Standard Expo SDK 54 App

The Baseline Stack

The source apps repeatedly converge on this Expo SDK 54 baseline. Optional packages vary by product, and each committed lockfile—not the semver ranges alone—records the dependency snapshot that was actually tested. See [Current Expo Baseline](#) for why that distinction matters.

```
{
  "expo": "~54.0.36",
  "react": "19.1.0",
  "react-native": "0.81.5",
  "expo-router": "~6.0.23",
  "react-native-reanimated": "~4.1.1",
  "@shopify/react-native-skia": "2.2.12",
  "react-native-svg": "15.12.1",
  "zustand": "^5.0.8"
}
```

Rule: deliberately pin the SDK line and commit the reviewed lockfile. Do not let a fresh `npx create-expo-app` silently move an established SDK 54 repository to a different generation.

The Default Folder Layout

This is the shape of a plain app with no native code. If you can build what you need inside this structure, do — everything past this point costs build time and debugging time.

```
MyApp/
  app.json           Expo config: name, bundle id, icons, plugins, permissions
  eas.json           build + submit profiles
  package.json
  .npmrc             project-specific npm policy, only when required
  tsconfig.json
  assets/
    icon.png         1024x1024, real PNG, no alpha
    splash.png
  src/
    app/             expo-router: file-based routes live here
      _layout.tsx
      index.tsx
    components/       presentational, no game loops
    constants/
    data/             static content, level definitions, word lists
    engine/           pure logic – no React imports, unit-testable
    services/         storage, audio, purchases, platform wrappers
    store/            zustand slices
    theme/
    types/
  scripts/           asset generation, validation, content audits
  tests/             engine and data tests, run before every build
  docs/             build reports, store setup notes
```

Two rules earn their keep:

- `src/engine/` **must not import React**. If the rules of the app can be tested without rendering anything, most bugs get caught before the simulator opens.
- `scripts/` **validates** `data/`. Every app with authored content has a script that fails the build if the content is malformed. That is cheaper than a rejected binary.

What Goes In `app.json`

Three identifiers/declarations are easy to forget until a submission stalls:

```
{
  "expo": {
    "ios": {
      "bundleIdentifier": "com.example.myapp",
      "buildNumber": "1",
      "infoPlist": {
        "ITSAppUsesNonExemptEncryption": false
      }
    }
  }
}
```

For the apps documented here, which used only exempt or operating-system-provided encryption such as ordinary HTTPS, `ITSAAppUsesNonExemptEncryption: false` was the appropriate declaration and avoided repeated export-compliance questions. Do not copy that value blindly into an app that vendors cryptography, implements a VPN, provides secure communications, or otherwise changes the encryption scope. Re-evaluate the declaration whenever the security architecture changes.

`buildNumber` must increase for every submitted build — see [App Store Connect Rejects Duplicate Submissions By Build Number](#).

What The App Store Requires

Apple will not tell you about most of these until a submission fails. This is the list that has actually blocked builds.

Privacy And Permissions

Every protected capability the app actually requests needs an accurate, human-readable usage description. Prefer the library's config-plugin option when one exists because it keeps generated native projects reproducible. Direct app config, checked-in native settings or a custom config plugin may be necessary for capabilities a library does not expose. Apple can reject vague descriptions; say what the app does with the data and do not request permission before it is needed.

```

    {
      "plugins": [
        ["expo-camera", {
          "cameraPermission": "Allow MyApp to access your camera to scan nearby
objects."
        }],
        ["expo-speech-recognition", {
          "microphonePermission": "Allow MyApp to hear your spoken questions.",
          "speechRecognitionPermission": "Allow MyApp to turn your speech into text."
        }]
      ]
    }
  }
```

The usage descriptions that have been needed across these apps:

Key	Needed when
<code>NSCameraUsageDescription</code>	any camera use, including hand tracking
<code>NSMicrophoneUsageDescription</code>	audio capture, voice input
<code>NSSpeechRecognitionUsageDescription</code>	Apple's Speech framework; a server-only transcription flow does not by itself require this key

Key	Needed when
<code>NSPhotoLibraryUsageDescription</code>	direct photo-library access; system pickers may provide scoped access without the broad permission
<code>NSLocationWhenInUseUsageDescription</code>	anything map or position based
<code>NSHealthShareUsageDescription</code> / <code>NSHealthUpdateUsageDescription</code>	HealthKit reads and writes

A privacy policy is mandatory for every app, whether or not it collects anything. Apple requires a link in App Store Connect metadata and an easily accessible link inside the app. An app that collects nothing still needs a page that says so and must accurately cover third-party SDK behaviour. Keep one source file per app, publish it at a stable HTTPS URL, and test both links before review.

Before The First Submission

- ☐ Bundle identifier registered and matching `app.json`
- ☐ App icon is a real 1024×1024 PNG with **no alpha channel**
- ☐ Current required iPhone screenshot set — **6.9-inch Dynamic Island** at the time of this edition — and a 13-inch iPad set **only if** the app declares tablet support. Verify Apple's current dimensions before capture. If the app is genuinely iPhone-only, disable tablet support instead (see [iPad Screenshot Requirement](#))
- ☐ Privacy policy URL live and reachable
- ☐ App Privacy questionnaire answered in App Store Connect
- ☐ `ITSAAppUsesNonExemptEncryption` declared
- ☐ Age rating completed
- ☐ Every permission string written in plain language
- ☐ In-app purchase products and agreements configured before review; include the first purchase with the appropriate app-version submission and test the product state with a sandbox/TestFlight account

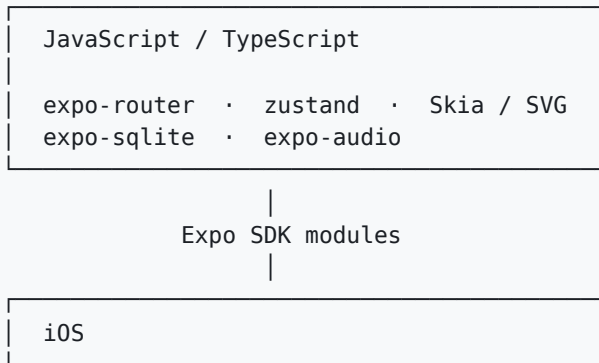
Encryption

In the apps covered here, standard HTTPS and operating-system cryptography were handled as exempt encryption, so `ITSAAppUsesNonExemptEncryption: false` was used. If you vendor cryptographic code or provide encryption as a primary function, the answer can change and documentation may be required. Treat this as an app-specific compliance decision, not a universal coding default.

When An App Strays From The Default

Most of these apps are plain Expo apps. Six of the named case studies are not, and each crossed the native boundary for a specific reason that the selected architecture could not solve reliably in JavaScript. This is the map of who deviates and why — the detailed lessons are in [Part 4](#).

The Default: No Native Code

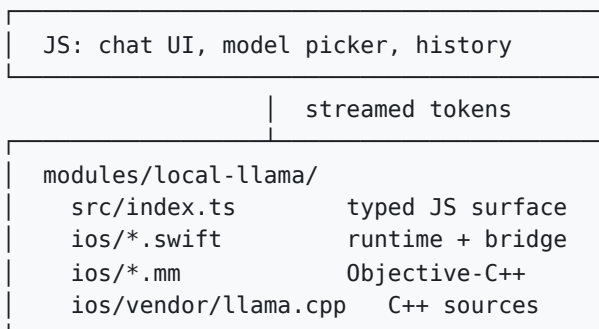


Can use Expo Go only when every native dependency is included in Expo Go's fixed runtime. Fast to iterate. No custom podspec or Swift in this layer.

Veranda, Melody Mind, Obsidian Wastes, Imperial Dreams, Junk & Jolt, Wayline, RealMath and Astral And Audio all live here. **If you can stay in this box, stay in it.**

Deviation 1 — On-Device Inference

Localresearch, QuestGiver



Why this implementation had to be native: the selected runtime was `llama.cpp`, which is C++, and Expo Go cannot load a custom native runtime. Other JavaScript or WebGPU inference approaches may exist, but they were not the tested architecture here. This implementation requires a development, EAS or locally compiled native build and must be proven on a real device.

What it costs: the vendored C++ must sit inside the module's `ios/` tree for CocoaPods to compile it, every **native runtime** change means a full rebuild, and you lose Expo Go for anything touching inference. Swapping a compatible GGUF that is downloaded at runtime does not itself

require rebuilding the binary.

Deviation 2 — GPU Particle Simulation

DotPhysics

```
JS: presets, interaction matrix, UI
```

```
| config in, stats out
```

```
modules/particle-life-metal/  
ios/ParticleLifeEngine.swift  
ios/*.metal      compute + render  
ios/ParticleLifeMetalView.swift
```

Why it had to be native: 20,000 particles at 60fps. Every particle pair interaction runs on the GPU in a Metal compute shader. Crossing the JS bridge per frame — even once — would have made the target impossible.

What it costs: touch input has to be handled natively too, because routing it through JS reintroduces the latency the native path existed to remove.

Deviation 3 — Large-Scale Battle Rendering

LongHorizon

```
JS: setup, HUD, results
```

```
| polling stats bridge
```

```
modules/long-horizon-metal/  
ios/LongHorizonShaders.metal  
ios/*.swift    MTKView renderer
```

Why it had to be native: thousands of units a side, simulated and drawn every frame. State lives in GPU buffers and never becomes JavaScript objects. The HUD polls for statistics rather than receiving per-frame updates.

What it costs: the JS side cannot inspect unit state directly, so debugging means native logging and a deliberate diagnostics surface.

Deviation 4 — Camera Hand Tracking

Three Finger Shoot, Make Army Go Away

JS: game loop, scoring, calibration UI

| classified gesture events

modules/gesture-detector/
ios/GestureDetectorView.swift
ios/GestureClassifier.swift
AVFoundation → Vision hand pose

Why it had to be native: Apple's Vision hand-pose API is a native Apple framework, exposed to Swift and Objective-C, and runs against the live camera buffer. The supported Expo/JavaScript path did not provide the capability and latency this mechanic required. Both games share the same module — it was written once and copied, which was the practical reuse boundary for these projects.

What it costs: a camera permission string, no Expo Go testing of the core mechanic, and per-user calibration to handle different hand sizes and lighting.

The Decision Rule

Go native only when one of these is true:

1. The selected capability or required performance is not available through the app's viable JavaScript libraries (for example Vision, `llama.cpp` or Metal).
2. Per-frame work would have to cross the JS bridge and cannot afford to.
3. The data is too large to serialise — GPU buffers, model weights, video frames.

Everything else belongs in `src/`. Each native module in this list added days of build debugging, and the reusable lessons are documented in Part 4.

Part 2 — Start Here

The short version. If you read nothing else, read this part before your next build.

Current Expo Baseline

For the source apps behind this guide, the proven baseline is **Expo SDK 54**. That is a project decision, not a claim that SDK 55 or later is inherently wrong.

As of 13 August 2026, Expo states that SDK 54 EAS builds default to Xcode 26 and therefore satisfy Apple's requirement for uploads to use the iOS 26 SDK or later. Apple evaluates the compiled binary and toolchain, not the label "Expo 54." Check the live requirements before each submission because both the EAS image and Apple's minimum SDK can change.

If starting another app in this family, or asking an agent to scaffold one, use the lockfile-validated SDK 54 stack unless there is a deliberate migration plan:

```
{
  "expo": "~54.0.36",
  "react": "19.1.0",
  "react-native": "0.81.5"
}
```

Use Node 20.19.4 or later within the Node 20 line used by the SDK 54 build images, and keep the exact patch versions recorded in the app's lockfile.

Rule: do not assume the newest Expo SDK is automatically the right choice for an established app. Prefer the repo-proven SDK 54 stack until a migration is explicitly approved, tested and reversible.

Key Takeaways

1. **Do not let non-critical audio block navigation or game state.** Fire and forget optional sounds; use a bounded wait only when sequencing truly matters.
 2. **Wait for Zustand persistence to hydrate** before rendering dependent UI.
 3. **Run** `npx expo-doctor` and the app's own tests before every release build.
 4. **Verify image content, dimensions and alpha**, not just file extensions.
 5. **Review the local EAS upload set.** Uncommitted files can be uploaded unless ignored; commit a reviewed snapshot for reproducibility, not inclusion.
 6. **Expo SDK 54 repos often need** `legacy-peer-deps` **when a demonstrated peer conflict remains.** Diagnose first; do not make it a universal npm default.
 7. **Declare only the device families the product actually supports.** If the app is iPhone-only, disable tablet support and verify the generated native config.
-

Pre-Build Checklist

Run the relevant checks **before every EAS build**:

```
# 1. Run project checks first (use the scripts the app actually defines)
npm run typecheck
npm test

# 2. Check Expo SDK and dependency alignment
npx expo-doctor

# 3. Verify image content rather than trusting the extension
file assets/*.png

# 4. Review exactly what is present locally and what ignore rules will exclude
git status --short
git diff --check

# 5. Build after reviewing and intentionally staging/committing desired changes
eas build --platform ios --profile production
```

EAS creates an upload archive from the local project using `.gitignore`, or `.easignore` when present. It can include uncommitted files. Inspect the archive with `eas build:inspect` when inclusion is uncertain. Avoid `git add .` in a generic checklist: stage reviewed paths so credentials and unrelated work are not committed accidentally.

Use `--auto-submit` only after the build profile, App Store record, identifiers, credentials and release metadata are confirmed. Building and submitting are different external actions and should remain separate when a sign-off is needed.

Starting a New App for App Store

When preparing a new app for App Store submission, use this prompt to set up a new chat. Keep `APP_STORE_GUIDE.md` as the full submission checklist and the app's `PRIVACY_POLICY_*.md` as the policy source of truth.

I'm preparing [APP_NAME] for App Store submission.

The app is located at: /path/to/[APP_FOLDER]/

I have already started the dev server locally (npx expo start --lan --clear).

Please help me prepare this app for production:

1. Review the codebase and identify any issues from IOS_APP_LEARNINGS.md
2. Run `npx expo-doctor` and fix any issues
3. Verify image assets are real PNG format
4. Check for any expo-av audio blocking issues
5. Check for Zustand hydration issues if using persist
6. Review app.json for production settings
7. Report the proposed build and submission commands, credentials/profile assumptions, and changed-file scope.
8. Stop for approval before triggering a paid cloud build or App Store submission.

Reference: /path/to/IOS_APP_LEARNINGS.md for project-specific history.

Part 3 — Running LLMs On The Device

Getting a real GGUF model running inside an Expo app, and keeping it offline afterwards.

Local LLM Setup With Expo And `llama.cpp`

Seen in: [Localresearch](#)

This was the pattern used to get the `Localresearch` iPhone app running a real GGUF model locally on-device while still keeping the wider app in Expo/React Native.

| What Actually Works

Important: `Expo Go` is useful for the app shell, but it does **not** run the local model runtime.

- `Expo Go` can be used for:
 - chat UI
 - model list / settings UI
 - downloading files into app storage
 - SQLite persistence

`Expo Go` cannot be used for:

- `llama.cpp`
- custom Swift / Objective-C++ native modules
- validating real on-device inference

Practical rule: Build the shell with:

```
npx expo start --lan --clear
```

Then validate real inference in a development/native iOS build first:

```
eas build --platform ios --profile development
```

Use a production build only for release-equivalent validation. Upload it to App Store Connect separately, or add `--auto-submit`, only when that release action is intentional and the store record is ready.

| Architecture That Worked

Seen in: [Localresearch](#)

Use Expo for the app shell, storage, and navigation/state, then add a local Expo native module for the model runtime.

Working split:

- JS / TS app:
 - chat UI
 - model picker
 - download flow
 - SQLite persistence
 - conversation history
- native module:
 - load GGUF from local path
 - call `llama.cpp`
 - stream tokens back into JS
 - unload / reload model when active model or context changes

Useful project structure:

```
Localresearch/  
  App.tsx  
  modules/  
    local-llama/  
      ios/  
        LocalLlamaModule.swift  
        LocalLlamaRuntime.swift  
        LocalLlamaBridge.h  
        LocalLlamaBridge.mm  
        LocalLlama.podspec  
        vendor/  
          llama.cpp/
```

Rule: Keep the vendored `llama.cpp` code inside the module's `ios/` tree if CocoaPods needs to compile it directly. Do not rely on podspec file globs that point outside the pod root.

Model Download And Storage Pattern

For this style of app, the model should be downloaded once and then used fully offline.

Pattern that worked:

- store GGUF files in the app documents/models directory
- store the local URI in SQLite
- keep model metadata in SQLite even if the local file is removed
- when removing a model, delete the file and clear the stored local URI
- when uninstalling the app, iOS deletes the app sandbox, including the models and SQLite DB

This gives you:

- built-in model options
- user-supplied Hugging Face GGUF URLs

- offline reuse after first download
- easy re-download later

Treat a user-supplied model URL as untrusted input. Require HTTPS, cap the declared and downloaded size, check available disk space, validate that the file is a supported GGUF before loading it, and prefer a known checksum for built-in models. Do not place access tokens in stored URLs or logs. Review each model's licence and redistribution terms before offering it inside a paid app.

| SQLite Tables That Were Useful

Keep at least:

- `models`
 - remote source URL
 - local URI
 - download size
 - display name
 - quantization

`conversations`

- title
- active model id
- folder id

`messages`

- conversation id
- role
- content

`settings`

- active conversation id
- active model id
- context window
- system prompt

Rule: Do not treat the filesystem as the source of truth. Treat SQLite as the source of truth and reconcile missing files on startup.

SQLite persistence is not automatically encrypted secret storage. If chats, documents or prompts are sensitive, define the threat model, use appropriate iOS file protection, avoid diagnostic logging of content, and document deletion and backup behaviour in the privacy policy.

| Seed And Upsert SQL Must Match Schema Exactly

Seen in: [Localresearch](#)

Problem: A TestFlight build crashed on first launch with:

```
Could not load Localresearch. Calling the 'prepareAsync' function has failed  
-> Caused by: Error code 1: 16 values for 17 columns.
```

Root cause: one of the SQLite `INSERT INTO models` statements listed `17` columns, but the `VALUES (...)` clause only had `16` placeholders. This kind of bug often hides in seed or upsert code and only shows up on a fresh install.

Solution: whenever you add or remove a column from a table, update every seed and upsert statement at the same time and count both the column list and placeholder list manually.

Practical check: test first-run database initialization on a clean install or cleared simulator/device data, not just an already-migrated local database.

Rule: for SQLite-backed mobile apps, schema changes are not finished until every insert/upsert path is audited for matching placeholder counts.

| `llama.cpp` Integration Lessons

| 1. Use A Native Expo Module, Not A Hacky JS Wrapper

The clean path is:

- Swift Expo module API surface
- a runtime helper in Swift
- Objective-C++ bridge for `llama.cpp`
- vendored `llama.cpp` sources compiled by CocoaPods

This gives a stable boundary:

- JS calls `loadModel`, `generate`, `interrupt`, `unload`
- native side handles all `llama.cpp` specifics

| 2. CocoaPods File Patterns Must Stay Pod-Local

Problem: Pod validation failed when `source_files` / `public_header_files` used absolute paths or referenced files outside the pod root.

Solution: Move vendored `llama.cpp` under:

```
modules/local-llama/ios/vendor/llama.cpp
```

and keep podspec globs relative.

Rule: If CocoaPods says file patterns cannot start with `/`, believe it and move the files rather than trying to outsmart the validator.

3. `ggml-metal.metal` Needed A Generated Embedded Variant

The Metal backend needed a generated embedded source during pod install.

The working pattern was:

- use `prepare_command` in the podspec
- generate an embedded `.metal` file
- generate an assembly file that embeds the Metal source/blob

Without this, the Metal backend setup was incomplete.

3a. `.incbin` Was Too Fragile For EAS Cloud Builds

Problem: EAS / Xcode builds could still fail even after the Metal embed path worked locally, because the checked-in generated assembly was still using:

```
.incbin "ggml-metal-embed.metal"
```

That leaves the assembler needing to find a sibling file at compile time. On remote EAS builders, local pod paths and caches are not stable enough to rely on that.

Symptom: pod install or archive failures where the Metal payload could not be found, even though the podspec had a prepare step.

Root cause: the local pod was still effectively depending on filesystem lookup during Xcode assembly, instead of shipping a fully self-contained assembly payload.

Solution: generate a self-contained `ggml-metal-embed.S` file made of `.byte 0x..` lines, so the assembly file contains the full Metal payload directly and no longer references any external path at build time.

Rule: for EAS-safe Metal embedding, do not rely on `.incbin` in the checked-in generated assembly. Emit a self-contained byte payload instead.

4. `GGML_VERSION` And `GGML_COMMIT` Were Missing

Problem: Xcode failed with undeclared identifiers like:

```
GGML_VERSION  
GGML_COMMIT
```

Root cause: upstream `ggml` often gets these values from CMake compile definitions, but CocoaPods was not supplying them automatically.

Solution: add the equivalent compile definitions in the podspec.

5. ARC And Metal Backend Files Did Not Mix By Default

Problem: EAS/Xcode failed with errors like:

```
'release' is unavailable: not available in automatic reference counting mode
ARC forbids explicit message send of 'release'
```

Root cause: vendored `ggml` Metal Objective-C files were being compiled under ARC, but the upstream code uses manual memory management.

Solution: enable ARC for your own bridge file(s), but compile the vendored Metal `.m` files as non-ARC.

Rule: Do not blanket-apply ARC settings to the whole pod when vendoring low-level Objective-C code.

6. Swift Import Of Objective-C Errors Can Become `throws`

Problem: Swift code tried to call Objective-C methods with explicit `NSError **` style arguments and failed with “extra argument `error` in call”.

Solution: when Swift imports those methods as throwing APIs, call them with `try`, not manual `error: &error`.

7. Swift 6 Sendability Checks Show Up Fast

The native module needed small Swift 6 cleanup:

- avoid capturing non-Sendable values carelessly in async closures
- return from generic continuation helpers correctly
- be careful with event emission closures

These were compile issues, not runtime issues.

8. `mtmd_tokenize` Rejected `bitmaps.data()` Due To C++ Const Strictness

Problem: Xcode failed in the multimodal bridge with:

```
no matching function for call to 'mtmd_tokenize'
```

Root cause: `mtmd_tokenize` expected the fourth argument as:

```
const mtmd_bitmap **
```

but the bridge was passing:

```
bitmaps.data()
```

from a:

```
const std::vector<const mtmd_bitmap *>
```

which produces:

```
const mtmd_bitmap * const *
```

That extra outer `const` is not implicitly accepted by C++ here.

Solution: prefer a correctly typed, non-const pointer-array container for the call rather than casting away the outer constness:

```
std::vector<const mtmd_bitmap *> bitmapPointers(
    bitmaps.begin(),
    bitmaps.end()
);

const int32_t tokenizeResult = mtmd_tokenize(
    mtmdContext,
    chunks,
    &inputText,
    bitmapPointers.data(),
    bitmapPointers.size()
);
```

If the exact upstream signature or ownership guarantee differs in your vendored snapshot, wrap it at the bridge boundary and document whether pointer entries may be modified. A C-style cast can hide a real const-correctness bug.

Rule: when bridging `std::vector<const T *>` into older C-style APIs, watch for `T * const *` versus `T **` mismatches. Fix the container/signature boundary instead of treating a const-stripping cast as a generic solution.

9. Expo `file://` URIs Must Be Normalized Before Passing To `PDFDocument`

Problem: PDFs imported from Expo / React Native looked valid, but native PDF loading failed when the bridge tried to open them.

Root cause: Expo often returns file URIs such as:

```
file:///var/mobile/Containers/Data/Application/...
```

Passing that string directly to:

```
[NSURL URLWithString:sourcePath]
```

is wrong, because `fileURLWithPath:` expects a plain filesystem path, not a URI. iOS can end up interpreting it as a malformed path like `file:///file%3A///...`.

Solution: normalize the incoming Expo path first by stripping or resolving the `file://` prefix, then create the native file URL from the resolved local path.

Rule: when bridging Expo file picker or photo/document URIs into native iOS APIs that expect paths, resolve `file://` URIs before calling `fileURLWithPath:`.

10. Large PDF Render Loops Need `@autoreleasepool` On iOS

Problem: Processing bigger PDFs could make the app terminate unexpectedly while converting pages into bitmaps or PNG data.

Root cause: `UIImage`, `NSData`, and other Objective-C objects created during each PDF page iteration were going into the autorelease pool and staying alive until the whole function returned. On multi-page PDFs, memory usage could spike high enough for iOS to kill the app.

Solution: wrap the body of each PDF page-processing iteration in:

```
@autoreleasepool {  
    // render page, create NSData/UIImage, preprocess, store result  
}
```

so those temporary objects are released after each page instead of at the very end.

Rule: a tight Objective-C/Objective-C++ loop that creates many autoreleased objects—such as large page/image render loops—should use an inner `@autoreleasepool` and measured memory bounds. Ordinary Swift loops that do not create that ownership pattern do not need this mechanically.

Recommended Native Runtime Behavior

Load only one model at a time.

Good runtime behavior:

- active model change triggers unload + reload
- context window change triggers unload + reload
- generation streams tokens into JS
- interruption is supported
- runtime stays unloaded if no local model is selected

For the tested phones, the first pass stayed deliberately small:

- text only
- one active model loaded at a time
- 2048 default context
- 4096 as an optional setting on devices that pass the memory check
- no tools, no RAG, no voice, no images

| EAS Build Workflow That Made Sense

Use this loop:

1. Build UI / storage logic in Expo locally.
2. Run:

```
npm run typecheck  
timeout 30s npx expo-doctor
```

3. Batch native changes.
4. Push and run one EAS iOS build.
5. Read the **first real compile error**, fix it, repeat.

This matters because many EAS failures are sequential:

- podspec validation
- missing vendored file paths
- missing compile definitions
- Swift import mismatches
- ARC issues
- then deeper compile / link issues

If you try to reason about all failures at once, you waste time.

| Best Practices For New Apps

- Start from Expo SDK 54 for these repos.
- Assume `Expo Go` is for shell work only when a local LLM is involved.
- Keep the native module self-contained under `modules/<module-name>/ios/`.
- Vendor the exact `llama.cpp` snapshot you tested.
- Store model metadata in SQLite and the actual GGUF in app documents storage.
- Offer built-in model choices plus a custom GGUF URL path.
- Check model license terms before shipping a paid app.
- Expect several EAS native build iterations the first time you wire `llama.cpp`.

Quick Checklist For Reuse In A New App

1. Scaffold Expo SDK 54 app.
 2. Add expo-file-system and expo-sqlite.
 3. Build the chat/model shell in JS first.
 4. Create a local Expo native module.
 5. Vendor llama.cpp inside the module iOS folder.
 6. Add podspec compile settings and Metal embed generation.
 7. Bridge load / generate / interrupt / unload.
 8. Persist model URIs, chats, settings, and folders in SQLite.
 9. Validate shell-only UI in Expo Go where compatible.
 10. Validate real inference, memory pressure, interruption and background/resume in a development/EAS build on every supported device class.
-

Part 4 — Native Modules: Swift, Objective-C++ And Metal

Where Expo stops and native code starts, and how to make the two meet without fighting the build.

Native Expo Module Development (Finger Shoot)

When building a native module using Swift for Expo:

Module Structure (Critical)

Prefer generating a local module with `npx create-expo-module@latest --local`, then keep the generated structure unless the project has a documented reason to depart from it. This is the hand-authored layout that worked in these SDK 54 projects; it is an observed structure, not the only structure Expo can discover:

```
modules/gesture-detector/
├─ expo-module.config.json  # "apple" was used by these modules
├─ package.json            # Package metadata and entry points
├─ src/
│   └─ index.ts            # TypeScript interface
└─ ios/
    ├─ ModuleName.podspec  # MUST be in ios/ folder
    └─ *.swift             # Swift files
```

expo-module.config.json

```
{
  "platforms": ["apple"],
  "apple": {
    "modules": ["GestureDetectorModule"]
  }
}
```

Current Expo module configuration accepts universal `"apple"` or granular `"ios"`, `"macos"` and `"tvos"` platform values. Use `"apple"` when the module is intentionally configured through the universal Apple block; use `"ios"` for an iOS-only module when that matches the current template. Keep the platform key and module block consistent rather than mechanically replacing one with the other.

Podspec Requirements

```
s.dependency 'ExpoModulesCore' # Capitalized, no hyphen!
s.source_files = "**/*.{h,m,mm,swift}"
```

Camera Permission in Native Code

Swift code must request camera permission BEFORE using AVCaptureSession:

```
func startTracking() {
    switch AVCaptureDevice.authorizationStatus(for: .video) {
    case .authorized:
        setupCamera()
    case .notDetermined:
        AVCaptureDevice.requestAccess(for: .video) { granted in
            if granted { self.setupCamera() }
        }
    case .denied, .restricted:
        // Emit error event to JS
        onGestureChange(["state": "error", "error": "camera_permission_denied"])
    }
}
```

Verifying Autolinking Detection

```
npx expo-modules-autolinking resolve --platform apple | grep gesture
```

Should show `podspecDir` pointing to your `ios/` folder.

Common Pitfalls

1. **Podspec not where the generated/configured module expects it** - in this local-module layout it lives in `ios/`
2. **Mismatched config key** - Keep `platforms` and the corresponding `apple / ios` module block consistent; this repo uses `apple`
3. **Wrong dependency name** - `ExpoModulesCore` not `expo-modules-core`
4. **No camera permission request** - Native code must call `requestAccess()`
5. **Changes need a native rebuild** - use a local `npx expo run:ios`, a development client, or EAS as appropriate; reloading Metro is not enough

Native Metal Particle Engine — iOS Learnings

Seen in: [DotPhysics](#)

1.

Expo Go cannot run this native module.

- DotPhysics uses a local Expo native module with `MTKView` and Metal.
- It needs a custom dev build, EAS build, or TestFlight build.

An earlier build path that compiled `.metal` sources during Xcode archive failed because the Metal toolchain was missing.

- Error: `cannot execute tool 'metal' due to missing Metal Toolchain`
- For that compiled-shader path, an EAS hook ensured this ran before Xcode: `xcodebuild -downloadComponent MetalToolchain`
- The later Swift-only runtime-source path did not compile `.metal` files in the archive and should not inherit this hook without evidence it is needed.

The app then built, but no particles appeared.

- Native stats did not arrive.
- The JS UI showed `waiting for native view` / `draw pending`.
- We added a staged diagnostic flow: `shell -> MTKView -> renderer -> config -> full run`.

The diagnostic build proved the native view path worked.

- The Expo native view mounted.
- `MTKView` mounted and had layout.
- Renderer creation failed before drawing.

Root cause: the renderer could not load the precompiled Metal library.

- Error: `Could not load ParticleLife Metal library from module bundle, main bundle, or default library.`
- The local Expo module build did not reliably provide a default `.metallib` in TestFlight.

Working solution for this app: compile embedded shader source at runtime.

- Do not rely only on a bundled/default `.metallib` for this Expo local module.
- Embed the shader source in Swift and compile it on the device with:
`device.makeLibrary(source: particleLifeMetalShaderSource, options: nil)`
- Keep bundled/default library loading only as fallback after runtime source compilation.

Native diagnostics that were useful:

- A `UILabel` inside `ParticleLifeMetalView`.
- A JS wrapper layout marker.
- Delayed `onReady` after `didMoveToWindow`.
- No `EventDispatcher` calls from native init.
- Staged buttons to mount shell, `MTKView`, renderer, config, and full native run.
- A native draw heartbeat for drawable/pass/command/uniform/particle state.

Safety fixes:

- `clearCellAggregates` needed an `aggregateCount` bounds guard.
- Swift and Metal uniform structs must match exactly.
- `MTKView` needed `isUserInteractionEnabled = false` so parent touch handlers receive touches.
- Renderer/config failure should update the native label and not crash the app.
- Apply Config and Full Native Run should be disabled until renderer creation succeeds.

Future recommendation for this architecture:

- Keep the staged diagnostics hidden behind a developer flag.
- Do not show diagnostics by default in the normal app flow.
- Use runtime shader compilation, or verify `.metallib` packaging in TestFlight before switching back to precompiled libraries.

LongHorizon Swift/Metal Battle Renderer - Additional Learnings

Seen in: [DotPhysics](#), [LongHorizon](#)

LongHorizon confirmed the DotPhysics approach still works for an Expo 54 iOS game with a local Swift/Metal native view, but it also exposed several extra failure modes that are easy to repeat if they are not written down.

1. Keep the local Expo module podspec Swift-only when using runtime shader compilation.

- The working pattern is: `s.source_files = '**/*.swift'`
- Do not add `.metal` files to `source_files` unless you deliberately switch back to an Xcode-compiled Metal pipeline and verify it in EAS/TestFlight.
- In LongHorizon, adding `.metal` sources caused Xcode/EAS failures such as `expected unqualified-id` and `expected expression`.

2. Treat embedded runtime shader source as generated code.

- Keep a readable `.metal` file as the editing source.
- Regenerate the Swift string file from that `.metal` file after every shader edit.
- Add a source check that verifies:
 - `LongHorizonShaderSource.swift` exactly matches `LongHorizonShaders.metal`
 - the renderer calls `device.makeLibrary(source: longHorizonMetalShaderSource, options: nil)` first
 - the podspec is not compiling `.metal` files separately

3. Avoid Metal keyword-like variable names in shader source.

- Runtime Metal compilation failed at the `Battle` checkpoint with: `program_source:443:24: error: expected unqualified-id`
- The failing line was: `OverlayVertex vertex = vertices[vertexId];`
- `vertex` is a shader-stage keyword in Metal, so using it as a local variable name is unsafe.
- Rename locals like this: `OverlayVertex overlayVertex = vertices[vertexId];`
- Practical rule: avoid local variable names such as `vertex`, `fragment`, and `kernel` in Metal shader code.

4. Stage native startup more finely than just "renderer".

- DotPhysics used shell -> MTKView -> renderer -> config -> full run.
- LongHorizon needed one extra split: shell -> MTKView -> probe -> battle -> config -> full run
- Meanings:
 - `shell`: React Native screen only
 - `MTKView`: native Expo view and Metal view mount
 - `probe`: tiny standalone Metal shader draws diagnostic geometry
 - `battle`: full renderer and full shader/pipeline creation
 - `config`: unit buffers and battle config are applied
 - `full run`: continuous simulation and drawing begin
- This turned an opaque crash into a visible shader compiler error.

5. Make one TestFlight build isolate multiple native stages.

- TestFlight uploads are limited, so do not ship a single all-or-nothing native start path while debugging.
- Add on-screen checkpoint buttons and ask the tester to record the last button tapped.
- If `Probe` passes but `Battle` fails, look at full shader/pipeline creation.
- If `Battle` passes but `Config` fails, look at buffer allocation, decoded config, uniforms, or unit data.
- If `Config` passes but `Full Run` fails, look at per-frame simulation, command buffers, and draw loop state.

6. Prefer a native label for diagnostics before trusting React events.

- A `UILabel` inside the native view is extremely useful because it can show native status without crossing the React Native event bridge.
- Do not emit `EventDispatcher` events from native init.
- Delay native event emission until after `didMoveToWindow`.
- During crash isolation, add a kill switch that disables all native events.
- If crash logs show `com.meta.react.turbomodulemanager.queue` and `ObjCTurboModule::performVoidMethodInvocation`, suspect native module/event invocation as well as the renderer.

7. Lazy-load the native view manager.

- Do not resolve `requireNativeViewManager(...)` during module import if the app can show a React shell first.
- Resolve it only when the user/tester presses the native-renderer load button.
- Wrap the native view in a React fallback so missing native registration becomes an on-screen error instead of an immediate render crash.

8. Keep audio isolated while debugging native rendering.

- LongHorizon crash logs included `AVPlayer` / `CoreMedia` activity near early crashes, so `expo-audio` was disabled with a source kill switch during native renderer diagnosis.
- Use type-only imports or guarded dynamic imports while audio is disabled.
- Re-enable background music and SFX only after the native renderer, config, and full run stages are stable.
- DotPhysics' simple music pattern is still a good reference, but do not mix audio startup with first-pass native renderer debugging.

9. Optional render effects must not block the core renderer.

- Projectile, arrow, overlay, and diagnostic pipelines should be optional where possible.
- If an effect pipeline fails, the unit renderer should still be able to start and show a native status message.
- Cap projectile/overlay draw volume so a polish effect cannot overwhelm the first playable build.

10. MTKView and layout safety rules still matter.

- Wait for non-zero bounds before creating the renderer.
- Set `MTKView.isUserInteractionEnabled = false` if React Native or the parent native view owns gestures.
- Keep Swift and Metal uniform structs exactly aligned.
- Do not assume `currentDrawable`, render pass descriptors, command buffers, or buffers exist; report missing draw prerequisites in the native label.

11. Validation commands for this pattern.

- Use local npm cache for package commands: `npm_config_cache=$PWD/.npm-cache`
- Run: `npm_config_cache=$PWD/.npm-cache npm run typecheck` `npm run doctor`
`npm_config_cache=$PWD/.npm-cache npx expo config --type public`
- Also run a small source check for:
 - embedded Swift shader equals `.metal`
 - runtime shader compilation is first
 - podspec stays Swift-only
 - known bad shader identifiers are absent
- Note: `expo-doctor` with an explicit `npm_config_cache` override may print a JSON parse warning even when `expo config --json --full` succeeds. In that case, run `npm run doctor` without the override and run `expo config` separately with the cache override.

12. Practical rule for future Swift/Metal Expo games.

- Start with the proven DotPhysics pattern.
 - Add LongHorizon's finer checkpoints immediately.
 - Keep runtime shader source generation deterministic.
 - Keep native events and audio off until the renderer stages are proven.
 - Only then remove or hide the diagnostic UI.
-

Part 5 — Real-Time Games: Skia, Reanimated And Simulation

Patterns for anything that runs a loop every frame instead of re-rendering on state change.

Game Development Patterns (Three Finger Shoot)

Seen in: [Three Finger Shoot](#)

| Preventing Game Loop Crashes (Multiple State Updates)

Observed symptom: duplicate collision handling caused repeated game-over transitions and side effects in one game loop; downstream native/UI work made the failure appear as an intermittent crash.

Root cause: several collisions in the same tick could all request the terminal transition before React committed the first state update. Multiple `setState` calls do not inherently crash React; the bug was duplicate transition work.

Solution: Use a ref to track game over state:

```
const isGameOverRef = useRef(false);

// In collision check:
if (collision && !isGameOverRef.current) {
  isGameOverRef.current = true; // Set immediately
  setPhase('gameover');        // Then update state
}

// Reset when starting new game:
const startGame = () => {
  isGameOverRef.current = false;
  setPhase('playing');
};
```

| Preventing Gesture Spam (Require State Reset)

Symptom: Users can "cheat" by holding a gesture (e.g., fist) to spam shoot.

Solution: Track if they've returned to neutral state:


```
// In gesture store
hasAimedSinceLastShoot: false,

triggerShoot: () => {
  const { hasAimedSinceLastShoot } = get();
  if (!hasAimedSinceLastShoot) return; // Can't shoot until aimed

  set({ hasAimedSinceLastShoot: false }); // Reset after shoot
  // ... fire shot
},

updateGesture: (state) => {
  if (state === 'aim') {
    set({ hasAimedSinceLastShoot: true }); // Ready to shoot again
  }
}
```

Random Values + useMemo (Preventing Flicker)

Symptom: Random text/values flicker rapidly on screen during animations.

Root cause: `Math.random()` in render = new value every re-render. Animated components cause frequent re-renders.

Solution: store a random choice for the lifetime that matters. State is the clearest choice when opening a modal/session; `useMemo` is acceptable only as a render optimisation when recomputation would not break semantics:

```
// ❌ BAD - flickers
const message = MESSAGES[Math.floor(Math.random() * MESSAGES.length)];

// ✅ Stable for one modal session
const [message, setMessage] = useState('');

const openModal = () => {
  setMessage(MESSAGES[Math.floor(Math.random() * MESSAGES.length)]);
  setVisible(true);
};
```

Validate App Store Links And Provide A Fallback

Seen in: [Melody Mind](#)

Observed symptom: In one app/device context, a direct App Store product URL did not open as expected from the app.

Fallback pattern used in this project: try the product page, catch rejection, then open the developer portfolio:

```
const productUrl =  
  'https://apps.apple.com/app/melody-mind/id6740667209';  
const fallbackUrl =  
  'https://apps.apple.com/au/developer/eoin-j-tolster/id1867338583';  
  
Linking.openURL(productUrl).catch(() => Linking.openURL(fallbackUrl));
```

Standard HTTPS URLs are supported by React Native `Linking`; a failure can be specific to the URL, simulator, storefront or transient system state. Catch the promise, retain a web/developer fallback, and verify the exact product link on a physical device. Do not generalise one failure into a ban on direct product URLs.

Non-Intrusive Cross-Promotion Pattern

Strategy: show promotions at a varied cadence within a repeating 29-game cycle. The gaps increase inside each cycle; the long-term frequency does not keep decreasing because the cycle restarts:

```
// Games: 3, 8, 16, 29, then restart cycle  
const intervals = [3, 5, 8, 13]; // Fib-like sequence  
const cycleLength = intervals.reduce((a, b) => a + b, 0); // 29  
  
shouldShowPromo: () => {  
  const position = totalGamesPlayed % cycleLength || cycleLength;  
  let promoPoint = 0;  
  for (const interval of intervals) {  
    promoPoint += interval;  
    if (position === promoPoint) return true;  
  }  
  return false;  
}
```

Why: the varied spacing feels less mechanical than every-N-games prompting. Track dismissal/conversion and cap the cadence; do not assume Fibonacci spacing is optimal for every audience.

Varied Movement Patterns for Game Objects

Problem: Straight-line movement is boring for shooting games.

Solution: Random movement pattern per object:

```

type MovementPattern = 'straight' | 'diagonal' | 'wave';

interface GameObject {
  movementPattern: MovementPattern;
  diagonalDirection: 1 | -1; // For bounce
  baseX: number;
  waveAmplitude: number;
  waveFrequency: number;
}

// In game loop:
switch (obj.movementPattern) {
  case 'straight':
    newX = obj.x + speed * dt;
    break;
  case 'diagonal':
    newX = obj.x + speed * dt;
    newY = obj.y + speed * 0.5 * dt * obj.diagonalDirection;
    // Bounce off edges
    if (newY < 0 || newY > maxY) obj.diagonalDirection *= -1;
    break;
  case 'wave':
    const phase = elapsed * obj.waveFrequency;
    newX = obj.baseX + elapsed * speed;
    newY = obj.baseY + Math.sin(phase) * obj.waveAmplitude;
    break;
}

```

Use seconds consistently for `dt` and `elapsed`, clamp or fixed-step after long pauses, and resolve boundary penetration rather than only reversing direction.

Native Gesture Detection (Apple Vision Framework)

Key learnings for hand tracking with `VNDetectHumanHandPoseRequest`:

1. **Fingertip detection is unreliable for fists** - Curled fingers hide tips under palm
2. **Use PIP joint comparison** - in the tested Vision coordinate transform, a fingertip at or below its PIP joint is treated as curled
3. **Debounce state changes** - Require multiple frames before changing gesture state
4. **Low confidence = hidden** - Treat low-confidence joints as curled

```

// This implementation's transformed Y axis treats a lower tip as curled.
let isCurled = tipY <= pipY

// Or if confidence is too low (finger hidden)
let isHidden = tipConfidence < 0.3
let fingerDown = isCurled || isHidden

```

Countdown Before Game Start

UX improvement: Show 3-2-1 countdown before gameplay begins:

```
type GamePhase = 'ready' | 'countdown' | 'playing' | 'gameover';

const startGame = () => {
  setPhase('countdown');
  setCountdownNumber(3);

  const timer = setInterval(() => {
    setCountdownNumber(n => {
      if (n <= 1) {
        clearInterval(timer);
        setPhase('playing');
        return 0;
      }
      return n - 1;
    });
  }, 1000);
};
```

Include: direction arrows, preview of obstacles, camera active so player can prepare.

The timer must be cancelled on unmount, restart and backgrounding so an old interval cannot move a replacement screen into play. Keep the interval handle in a ref or effect cleanup rather than relying only on the closure above.

Large Skia Campaign Case Study — August 2026

Source: an anonymised personal Expo SDK 54 iPhone project.

Recent physical-device passes exposed several problems that source-only checks either missed or described incorrectly. The patterns are useful diagnostic leads for real-time Expo/React Native games using similar Skia, Reanimated and Expo Router versions. The lease, cache size, disposal timing and mapper budgets below are the proven contract for this project—not universal Skia API guarantees.

A Skia Picture Recording Is Live Native Work, Not A Harmless Data Build

One mountain-arena crash was an `EXC_BAD_ACCESS` race between two native operations: the gameplay Canvas was presenting a Metal frame while another Skia reconciler was still recording the next arena's canopy picture.

Three individually reasonable decisions combined to cause it:

- ground and canopy recordings were started together
- only the ground recording was awaited
- readiness meant “ground exists,” not “all recording has finished”

That arena exposed the race because it had an unusually expensive canopy layer. The durable fix was a two-sided presentation coordinator, not a different promise order:

- every live Canvas holds a shared presentation lease
- picture recording holds an exclusive lease
- recordings are serialized, including recordings for different arenas
- both ground and canopy are awaited before an arena is ready
- a Canvas requested during recording waits before mounting
- a failed layer gets a bounded retry and then a cheap fallback

Rule: nothing should record into Skia while any Canvas can present that native drawing state. Enforce this structurally at the shared Canvas boundary.

| Expo Router Focus And Component Mounting Are Not The Same Lifecycle

`router.push('/play')` can leave Home mounted under the play route. If the Home Canvas held a presentation token for its whole component lifetime, it would block arena recording after the player had visibly left Home.

The correct lifetime is route **focus**, not React mount lifetime. Covered screens should release presentation leases and stop decorative clocks, animations and other recurring work.

Rule: in an Expo Router app, never assume a covered route has unmounted. Use focus-aware resource ownership for Canvases, timers and animations.

| Removing A JavaScript Reference Does Not Prove Native Memory Was Freed

The arena-picture cache was bounded in JavaScript, but deleting an entry only dropped the JS reference. The native `SkPicture` could remain alive until JSI garbage collection decided to reclaim it.

The safer policy is:

- retain only the current and next arena per layer
- explicitly call `SkPicture.dispose()` when evicting an older arena
- dispose only while the exclusive presentation lease proves no Canvas is using the picture
- on return to Home or campaign completion, unmount gameplay first, clear both arena layers, and navigate only after disposal finishes
- keep small, repeatedly reused character and UI images cached

Clearing everything after every level would create more decoding and recording churn and make transitions slower. A two-arena working set gives the next level fast entry without retaining the whole campaign. Forced garbage collection is not a dependable public React Native strategy.

| Loading UI Must Report Real Work

The long pause between levels became understandable once the crossing showed the two tasks actually gating entry:

1. PREPARING GROUND
2. PREPARING SCENERY
3. NEXT PATH READY

Continue stays disabled until both layers are ready. The completed state remains visible so the player sees that the wait has ended. Fast direct entry can still use a short anti-flicker delay, but an actual between-level wait should not hide its status or display an invented percentage.

Prewarming should begin at the earliest safe moment, such as when the current level's exit opens. Once the crossing begins, the completed simulation should unmount rather than continue consuming CPU behind an overlay.

Rule: a progress bar should be driven by completed operations, not elapsed time. If only two operations are measurable, show two honest stages.

| Display Refresh Rate Can Silently Double Game Work

`useFrameCallback` follows the device display cadence. Publishing a cloned fixed-pool world on every callback meant a 120 Hz ProMotion phone performed twice the intended React/mapper invalidation work.

The game now measures every display callback but accumulates simulation time and publishes the world at no more than 60 Hz. The callback also stops publishing while the route is covered, iOS is inactive, the game is paused, a terminal recap is open, or a frame error has stopped play. Fractional carried time is discarded across those lifecycle boundaries so resume does not replay hidden work.

Rule: distinguish display callbacks, simulation steps and React/state publication. They do not need to run at the same frequency.

| Read The Analytics Report Type Before Diagnosing A Crash

One supplied later-level file was a `bug_type: 202` CPU-use diagnostic, not a crash report. It showed high CPU use and a growing footprint, but also said that iOS took no action. It contained no exception, signal, termination reason or faulting thread, so it could not prove a crash or memory leak.

For a disappearance, collect the report that corresponds to that event:

- a crash report with the exception, signal and faulting stack
- a jetsam report if iOS terminated the app for memory pressure
- CPU diagnostics as supporting performance evidence, not a substitute

Rule: never infer a native crash mechanism from a CPU diagnostic merely because both occurred during the same level. Match the timestamp and report type first.

| Source Gates, Exports And Device Proof Answer Different Questions

TypeScript, lint, unit tests, mapper validators and `expo export` can prove source contracts and packaging. They cannot prove:

- native memory settles after repeated transitions
- 1,110 mounted mappers hold frame rate on a phone
- contrast is readable on the physical display
- a 44-point control feels comfortable under a thumb
- a Skia/Metal race no longer occurs on the affected route

Keep the claims separate. After lifecycle or rendering changes, repeat the exact cold-launch device routes that exposed the problem and test long enough to observe heat and memory trends.

| Performance Counters Must Follow The Mounted Component Tree

The original mapper counter counted top-level component functions but did not resolve nested mounts or repeated children. Five `EffectSlot` instances were reported as five mappers even though their nested `SpellImpact` and particle layers made them 405. The true synthetic maximum was 1,110, not 710.

A useful counter must:

- recursively resolve mounted child components
- account for array/map repetition
- guard against cycles and accidental double counting
- distinguish mounted mappers from continuously dirtied values
- keep synthetic worst-case counts separate from real per-level workloads

Rule: fix the measuring tool before optimizing to its budget. A precise budget based on an incomplete tree is still wrong.

| A Control Drawn Outside Its Parent May Be Visible But Untappable

Several UI failures shared the same geometry problem: a tall control was placed inside a shorter clipped strip. It remained visible, but React Native could not reliably deliver touches outside the parent's bounds. This affected TALK, a passage action and the campaign's ending buttons. A production-only banner offset also overlapped Pause even though the development layout looked safe.

Reusable rules:

- the complete hit target must fit inside its parent frame
- use genuine 44×44-point targets for important iPhone actions
- `hitSlop` does not rescue a target clipped by its layout parent
- test production and development geometry, not only the visible dev layout
- assert frame arithmetic for fixed-height strips

- move a large decision to its own overlay instead of forcing it into a small status rail

For dense landscape screens, pin the required decision and Continue action. Optional shopping can wrap or scroll, but it must not hide the choice that gates progress.

| Transparent Equipment Layers Must Preserve The Base Sprite

Upgraded companions appeared as empty suits of armour because a Skia `color` blend turned every pixel in the sparse equipment Atlas cell opaque. The overlay then covered the companion underneath.

The repair was to use alpha-preserving `modulate`, draw the base actor first, and refuse to draw raster equipment unless the base image loaded. If the base decode fails, use the procedural actor and equipment fallbacks together rather than drawing equipment alone.

Rule: an optional overlay must never become the only visible layer. Test alpha retention, base-first ordering and failure behavior.

| Floating Sprites Usually Mean The Source Frame And Runtime Anchor Disagree

Several terrestrial monsters did not need their shadows moved. Their painted feet sat too high inside transparent sprite cells while runtime placement correctly treated the cell floor as the ground.

The durable fix was to ground-align the source frames and add a per-frame art validation contract for creatures that require ground contact. Moving the shadow to match a badly padded image would only make collision, feet and shadow disagree in a different way.

Rule: define one sprite ground line and validate every terrestrial frame against it. Treat shadows as evidence of a bad anchor before changing shadow math.

| Route Objectives Need A Forward-Path Simulation

Several level problems were data-correct but play-wrong:

- all enemies spawned near the beginning, leaving a long empty route
- sleeping enemies sat outside the natural wake radius and the guide sent the player backward
- an exit quota was lower than the authored roster and opened early
- holding right could reach an exit without engaging later encounters

The reusable checks are:

- distribute encounters in authored knots along the path
- make completion quotas agree with the required one-life roster
- point toward the nearest sleeping forward encounter when nothing is awake
- simulate the expected player route and verify every required post wakes
- add a no-cast/direct-exit regression so passive completion stays impossible

- reuse bounded enemy slots sequentially when a level needs more total bodies without increasing its simultaneous fixed pool

Rule: validate the route the player will actually walk, not just whether every coordinate is legal in isolation.

Reanimated, Simulation And Layout Case Study — July-August 2026

The entries above came from device passes in August. These came earlier and were expensive because ordinary Node, TypeScript, lint and unit-test runs did not exercise the worklet transform, Skia or GPU paths. Two worklet-call defects silently aborted the affected device build; the invalid keyframes reached the route error boundary; and render-time shared-value access was a separate correctness/performance violation. Each now has an appropriately scoped static gate, verified by re-introducing the original defect and watching that gate fail.

| A Reanimated Worklet May Only Call Other Worklets

Symptom: the app vanishes to the home screen. No red box, no error in Metro, no JavaScript stack. Often several seconds into a screen rather than on mount.

Root cause: a `'worklet'` function called a plain JavaScript function. Calling a non-worklet from the UI thread throws — and a throw there is not a red box. Reanimated evaluates worklets from a `CADisplayLink` on the main thread, so the exception escapes into C++, `std::terminate` runs, and **iOS aborts the process.**

```
function nearestSpeakingFigure(...) { ... }           // plain function

export function createHudSnapshot(world) {
  'worklet';
  return nearestSpeakingFigure(world);                 // ← aborts the process
}
```

Three functions shipped with this bug in one project. Marking the callee `'worklet'` is the whole fix; the danger is that nothing local tells you the callee is not one.

Rule: write a static validator that walks every `'worklet'` function and proves each identifier it calls is also a worklet. Do not rely on review — the call looks completely ordinary at the call site.

| Worklets Capture Identifiers At Module Evaluation, Not At Call Time

Symptom: the same silent process abort as above, but only on device. Node passes, because ordinary function declarations hoist there.

Root cause: a worklet that calls a function declared *below it in the same file* captures `undefined`. The worklet's closure is built when the module is evaluated, not when the worklet runs.

```
export function stepWorld(world) {
  'worklet';
  return applyBoonEffect(world); // captured as undefined on the UI thread
}

function applyBoonEffect(world) { 'worklet'; ... } // declared too late
```

Rule: enable `@typescript-eslint/no-use-before-define` for every directory containing worklets and never disable it locally. The fix is always to move the declaration up, never to silence the rule.

Reanimated CSS Keyframe Selectors Must Be Percentages Or 0-1 Numbers

Symptom: a screen dies through the error boundary the instant an animated element renders — `Invalid keyframe selector`.

Root cause: keyframe objects authored CSS-style with bare numbers. Reanimated accepts a number **between 0 and 1**, or a percentage **string**. `45` is neither.

```
// Throws on render
const KEYFRAMES = { 0: { opacity: 0 }, 45: { opacity: 1 }, 100: { opacity: 0 } };

// Correct
const KEYFRAMES = { '0%': { opacity: 0 }, '45%': { opacity: 1 }, '100%': { opacity: 0 } };
```

Five of these shipped in one pass. Two fired within seconds of starting the first level; the other three were latent behind low health, a hit, and an NPC speaking — so a smoke test would have found two of five.

Rule: validate keyframe selectors statically. Latent animation states are not reachable by casual play, so "it ran for a minute" proves very little.

Never Read A Shared Value During React Render

```
const elapsed = useSharedValue(world.value.time); // reads .value in render
```

Shared values live on the UI thread. Reading one during render is a cross-thread read at an undefined moment; it also silently seeds state from whatever the frame happened to hold. Seed from a constant and write the real value from an effect or a worklet.

Rule: `.value` belongs in worklets, `useDerivedValue` and effects — never in a render body. This is cheap to check with a regex and worth gating.

Live Skia Scene Complexity Still Costs Per Frame

Symptom: the app was killed on launch as scenery detail grew.

Root cause: static scenery built as retained Skia components — about **760 live nodes per arena** on top of ~620 for gameplay. Nothing moved, but every node was still visited each frame.

Recording that same component tree once with `drawAsPicture` and replaying it as a single `<Picture>` node sharply reduced retained scene-graph and reconciliation work. Replay still consumes draw/GPU time and native memory; it is cheaper in this case, not free.

Two things that matter when doing this:

- **Record the real component tree.** An earlier attempt hand-ported each shape into imperative canvas calls and immediately drifted — it covered ten of twenty clutter kinds and six of seventeen landmark kinds, silently drawing everything else as a generic pebble. Recording the existing tree makes that class of mistake impossible.
- Recording is live native work with its own hazards — see entry **1** in the section above, which is the other half of this lesson.

Rule: static groups are strong candidates for a recorded picture. Measure frame time, GPU work and native memory before and after; keep dynamic or state-dependent nodes live.

| Anything Derived From A Runtime Dimension Must Be Bounded

Arena sizes doubled, and every blur sigma derived from a dimension doubled with them — reaching **sigma 328 on a 3432-point rectangle**, which is enormous GPU work for an effect nobody could perceive past about 48.

The same applies to radii, particle counts, offscreen surface sizes and cache budgets. A formula that was reasonable at one content size is not a constraint.

Rule: every value computed from a world/screen dimension gets an explicit cap next to the formula, not a comment promising the input stays small.

| Clamp The Frame Delta Before Stepping A Simulation

Symptom: enemies occasionally walked straight through the player, or through a wall, after a hitch — a stall, a recording, a screenshot, returning from background.

Root cause: the raw frame delta was passed to the simulation step. Collision was position-based rather than swept, so a single 0.6-second step teleported every actor past whatever it should have hit.

```
const FIXED_STEP_SECONDS = 1 / 60;
const MAX_CATCH_UP_SECONDS = 0.10;

// Drop time from a long pause/background transition; do not replay it as combat.
accumulator += Math.min(Math.max(frameDeltaSeconds, 0), MAX_CATCH_UP_SECONDS);

let steps = 0;
while (accumulator >= FIXED_STEP_SECONDS && steps < 6) {
    stepWorld(FIXED_STEP_SECONDS);
    accumulator -= FIXED_STEP_SECONDS;
    steps += 1;
}
```

Reset the accumulator when the app backgrounds, a route becomes covered, or play pauses. If large steps are a legitimate mechanic, use swept collision rather than relying only on a clamp.

Rule: use bounded fixed-step catch-up and discard long inactive gaps. Position-based collision and unbounded deltas cannot both be correct.

React Native Layout Defaults Are Not The Web's, And Layout Bugs Are Arithmetic

A between-levels screen shipped with a truncated label (WAND FOC...) and a progress track drawn past its own panel border — on a screen that was two-thirds empty. Every cause was a number, not a taste question:

- a fixed **62-point** label at **9-point** text could not hold WAND FOCUS
- label + gap + eight pips + gaps + padding came to 213 points inside a 176-point card, so the track simply overflowed
- `flexShrink` defaults to **0** in React Native, not **1** as on the web, so a wrapping row decides how many items fit from `flexBasis` alone. Two cards at `flexBasis: 288` missed fitting an iPhone 15 in landscape **by one point** and stacked into a single column on a screen with room for both

The durable fix was to make the arithmetic a test rather than trusting it:

```
const rowWidth = labelWidth + rowGap + PIPS * pipWidth + (PIPS - 1) * pipGap;
assert.ok(rowWidth <= columnWidth - cardPadding - border);
```

Rule: set a minimum text size and hold every screen to it, then assert that fixed-width rows fit their containers. Landscape phones are narrower than they look once safe-area insets and padding are subtracted — compute the worst case for the smallest device you support.

A Test That Pins Source Text Can Hold A Bug In Place

Source-shape tests are genuinely useful for architecture contracts — they pin JSX order, dependency arrays and style values that no type system checks. They have one failure mode worth knowing about: **two tests asserted the broken source literally**, one pinning a bad keyframe selector and one pinning a shared value read during render. The suite was not failing to catch those bugs, it was actively protecting them.

When a fix touches something a source test pins, update the assertion to the new invariant **and state why in the test**. Never delete it to get green, and never copy the current source into an assertion without asking whether the current source is correct.

Rule: a source-shape assertion should encode the rule, not a snapshot. If you cannot write down why the pinned text is right, do not pin it.

| A Glitch At The Same World Position Usually Means A Streaming Boundary

Symptom: while walking through a large Skia map, the whole scene briefly glitched or flickered. It initially looked movement-related, but it could be reproduced by walking forward and backward across one exact point after a camp. It did not happen simply because the player was moving, standing still, near water, or when the weather changed.

That distinction matters:

- a weather or animation fault normally follows time
- a camera-aliasing fault normally follows sub-pixel movement continuously
- a fault that repeats at the same world coordinate points toward a spatial partition, trigger, level-of-detail boundary, or streamed chunk swap

The frame meter reached 33.3 ms, but there was no large enough recurring spike to explain the location-specific behaviour by frame rate alone.

| The First Plausible Fix Was Real, But Incomplete

The first investigation found that some 256-point tiles were being stretched to 257 points to hide seams, even though their atlas frames had no bleed. The camera lead also reintroduced fractional movement after the camera had been pixel-aligned. Both were genuine defects.

The atlases were rebuilt with three pixels of edge bleed and rendered at exact scale, and the camera lead was smoothed and pixel-aligned. The correct atlas geometry is:

```
const TILE_SIZE = 256;
const BLEED = 3;
const FRAME_SIZE = TILE_SIZE + BLEED * 2; // 262

const sprite = Skia.XYWHRect(frame * FRAME_SIZE, 0, FRAME_SIZE, FRAME_SIZE);
const transform = Skia.RSXform(1, 0, x - BLEED, y - BLEED);
```

This removed resampling and a camera snap, but the original location-specific glitch survived. A good diagnosis must change when a clean experiment disproves it; do not keep tuning a theory merely because the defects it found were real.

| The Campaign-Wide Cause

The maps streamed a 3×3 neighbourhood of 2,048-point chunks selected with raw `Math.floor` boundaries. Several authored routes ran close to grid corners. A short diagonal movement could therefore change X and then Y almost immediately, causing two large Skia tree commits close together. An audit found this shape on many authored routes; one later route crossed the two raw boundaries only 3.8 world points apart.

The chunk coordinates were also passed into a large memoized canvas component. At a boundary, the renderer rebuilt aggregate arrays and native display-list inputs for all nine resident chunks, even though six chunks were unchanged during an ordinary horizontal or vertical transition.

| The Durable Fix

Use one shared **two-dimensional rectangular dead band** for every streamed map. Keep the current chunk while the player remains within the current chunk extended by a small presentation margin. Once the player leaves that rectangle, derive both axes from the current position in one atomic update:

```
const HYSTERESIS = 256;

const insideExtendedChunk =
  x >= current.x * CHUNK_SIZE - HYSTERESIS &&
  x < (current.x + 1) * CHUNK_SIZE + HYSTERESIS &&
  y >= current.y * CHUNK_SIZE - HYSTERESIS &&
  y < (current.y + 1) * CHUNK_SIZE + HYSTERESIS;

if (insideExtendedChunk) return current;
return chunkForPoint(x, y); // Recompute X and Y together.
```

Do not give X and Y independent hysteresis states. Near a corner, independent states can still turn one diagonal crossing into two commits.

Then render the resident window as stable, memoized physical chunks keyed by coordinate:

```
{residentChunks.map((chunk) => (
  <GroundChunk key={` ${chunk.x}:${chunk.y}`} chunk={chunk} />
))}
```

With coordinate identity, a one-axis move retains six of nine children and changes only the entering row or column. A diagonal move retains four and changes five, but it still happens as one deliberate presentation update. Keep tile arrays, scenery ownership and paths cached in small bounded caches; do not retain decoded art for the whole campaign.

Tall scenery needs one additional precaution. Preserve a small fringe or silhouette margin outside the physical owner chunk so a tree canopy or building does not pop into view merely because its ground anchor has not entered the resident window yet.

| Atlas Reuse Must Share Geometry, Not Just The PNG

The campaign-wide review found another latent variant: one level reused the new 262-pixel bled ground and water atlases but still sliced them as 256-pixel frames and drew them without the three-point negative offset. That can sample across neighbouring frames and recreate tile corruption even when chunk streaming is stable.

Rule: if two maps share an atlas, they must also share its frame size, bleed, sampling mode and destination offset. Test renderer geometry against the art manifest; checking only the image dimensions or frame count is not enough.

| How To Prove This Class Of Fix

1. Record the exact world coordinate and walk across it in both directions.
2. Temporarily pin the presented chunk; if the glitch disappears before the resident floor runs out, the streaming boundary is implicated.
3. Simulate every authored route at small distance increments and record chunk transitions. Assert that no two presentation swaps occur inside the chosen hysteresis distance.
4. Assert that the union of keyed physical chunks exactly reproduces the old aggregate resident floor and scenery set, with no duplicates or omissions.
5. Assert that an adjacent one-axis window retains six coordinate keys.
6. Check source frame geometry, bleed and transforms for every atlas consumer, including levels that reuse another level's PNG.
7. Run typecheck, lint, source validators, the full test suite and a clean iOS export.
8. Finally, repeat the exact boundary on a physical iPhone. Automated route and export proof shows that the architecture is consistent; it does not prove that the visible device glitch has disappeared.

Rule: location-repeatable glitches deserve a spatial-boundary audit before weather, random effects or general frame-rate tuning. Stabilize selection and retain renderer identity; do not merely make the full boundary rebuild faster.

Different failure classes need different evidence. Items 15–16 caused silent device aborts in the affected build. Item 17 produced a route-level keyframe error, while item 18 is a render-time thread/correctness violation. If a tester says “it just closed,” start with the worklet-call graph and native crash report; still keep all four static checks because casual play does not cover every state.

Short Checklist For Future Skia Game Passes

1. Are all Canvases focus-aware and using the shared presentation boundary?
2. Can picture recording overlap a presenting Canvas in either ordering?
3. Are both ground and canopy complete before readiness becomes true?
4. Are evicted native pictures explicitly disposed at a safe lifecycle point?
5. Does the cache retain only the working set rather than the full campaign?
6. Does loading progress represent real completed operations?
7. Are hot callbacks capped independently of a 120 Hz display?
8. Do covered/background/paused routes stop recurring work?
9. Does every important touch target fit inside its parent at 44 points?
10. Do sprite overlays preserve alpha and require a visible base layer?
11. Do terrestrial sprite frames meet the shared ground line?
12. Have the exact affected routes been repeated on a physical iPhone?
13. Is the supplied analytics file actually a crash or jetsam report?
14. Are source/export claims clearly separated from device evidence?
15. Does every worklet call only other worklets?
16. Is every worklet's callee declared above it in the file?
17. Are all keyframe selectors percentage strings or 0-1 numbers?
18. Are shared values read only in worklets, derived values and effects?
19. Is all static geometry recorded into a picture rather than retained?
20. Does every dimension-derived blur, radius or count have an explicit cap?
21. Is the simulation delta clamped before the step?
22. Do fixed-width rows fit their container on the smallest supported device?
23. Does every updated test assertion encode the rule rather than the new source?
24. Do streamed maps update both chunk axes atomically behind one 2D dead band?
25. Are resident chunks keyed by coordinate so overlapping chunks retain identity?
26. Does every atlas consumer match the manifest's frame size, bleed and offset?

Part 6 — Production Crashes And Their Causes

Failures that only appear in a release build, on a real device, or on a clean install.

Critical Production Issues

Seen in: [My Pet Slime](#), [Imperial Dreams](#)

| Zustand Persist Hydration Timing

Symptom: App works in Expo dev, but in TestFlight nodes appear locked after completing levels.

Root cause: In the affected stores, Zustand persist used an asynchronous storage adapter. The tree rendered before saved progress loaded. A synchronous adapter has different timing, so confirm the configured storage before applying this gate everywhere.

Solution:

```
// progressStore.ts - Add hydration tracking
_hasHydrated: false,
setHasHydrated: (state) => set({ _hasHydrated: state }),

// In persist config:
onRehydrateStorage: () => (state) => {
  state?.setHasHydrated(true);
}

// In components - Wait for hydration:
const { _hasHydrated } = useProgressStore();
if (!_hasHydrated) return <ActivityIndicator />;
```

| An Awaited `expo-av` Operation Stalled A Production Flow

Observed symptom: in affected TestFlight builds, an audio promise did not settle promptly, so code sequenced after it—navigation, completion or input handling—never ran.

Root cause: `await` suspended that async control path. It did not block the JavaScript event loop or “everything,” but too much product logic depended on the audio operation completing. `expo-av` is also deprecated and unmaintained in SDK 54, which makes migration preferable to expanding new usage.

Solution: fire and forget non-critical UI sounds, with explicit error handling:

```
// Fragile when the action should not depend on sound completion
await playButtonSound();
onPress();

// The action continues even if optional sound fails
void playButtonSound().catch(reportNonFatalAudioError);
onPress();
```

When sequencing genuinely matters, use a bounded helper and clean up the timer:

```
async function withTimeout<T>(operation: Promise<T>, milliseconds: number) {
  let timer: ReturnType<typeof setTimeout> | undefined;
  const timeout = new Promise<never>((_, reject) => {
    timer = setTimeout(() => reject(new Error('Audio timed out')), milliseconds);
  });

  try {
    return await Promise.race([operation, timeout]);
  } finally {
    if (timer) clearTimeout(timer);
  }
}
```

`Promise.race` does not cancel the underlying native operation. If the API offers abort/unload semantics, invoke them after timeout; otherwise guard against late completion and release the player during cleanup.

Rule: optional sound must not gate core state. Await loading, recording, cleanup or deliberate sequencing when required, but give native operations a bounded failure path.

A React Native Modal Crashed In These iOS Builds

Observed symptom: In specific release builds covered by this guide, presenting a particular `<Modal>` path sent the app to the home screen without a JavaScript error. React Native `Modal` is a supported component; this is not evidence that all iOS modals are unsafe.

Workaround used: replace that overlay with an absolute-positioned `<View>` so it remains inside the current React Native view hierarchy:

```
// Crashed in the affected app/path
<Modal visible={true} transparent={true}>
  <MyScreen />
</Modal>

// Stable workaround in that app
{showScreen && (
  <View style={{ position: 'absolute', ...StyleSheet.absoluteFillObject, zIndex: 1000 }}>
    <MyScreen />
  </View>
)}
```

Before adopting the workaround globally, collect the native crash report and test whether presentation style, navigation timing, nested modals, orientation or the modal's content is the real trigger. An absolute overlay has different accessibility, focus, back-button and native-presentation semantics and must be tested accordingly.

| expo-audio Playback Status Can Cause Infinite React Update Loops

Symptom: Terminal fills with `Maximum update depth exceeded`, often pointing at a root audio provider or `_layout.tsx`. The app may freeze or constantly rerender after a music track ends.

Root cause: Using `musicStatus.didJustFinish` inside a `useEffect` that also calls `setState(...)` can loop if the "finished" status survives across rerenders. Guarding by `trackIndex` is not enough because the same finished player can keep retriggering the effect.

Solution: Guard by the unique audio player/status id and only advance the playlist once per finished player instance:

```
const handledFinishedPlayerIdRef = useRef<number | null>(null);

useEffect(() => {
  if (!musicStatus.didJustFinish) {
    return;
  }

  if (handledFinishedPlayerIdRef.current === musicStatus.id) {
    return;
  }

  handledFinishedPlayerIdRef.current = musicStatus.id;
  setTrackIndex((currentIndex) => (currentIndex + 1) % BACKGROUND_TRACKS.length);
}, [musicStatus.didJustFinish, musicStatus.id]);
```

Rule: If an audio status effect sets React state, always make sure the trigger is uniquely consumed once. Do not rely on `trackIndex` alone to break the loop.

| Expo Swift Import Access-Level Mismatch During EAS Build

Symptom: EAS iOS build fails in the `Run fastlane` / Xcode step with:

```
ambiguous implicit access level for import of 'Expo'; it is imported as 'internal' elsewhere
```

Often the failing file is the generated `ExpoModulesProvider.swift`, even though the actual bad import is in your app's checked-in native code.

Root cause: The native iOS project had a nonstandard Swift import in `ios/<AppName>/AppDelegate.swift`:

```
internal import Expo
```

Expo's generated Swift files import `Expo` normally:

```
import Expo
```

On newer Xcode / Swift toolchains, mixing those can cause an access-level ambiguity during archive builds on EAS.

Solution:

```
// ios/<AppName>/AppDelegate.swift
import Expo
```

Then rerun:

```
npx expo-doctor
```

If `expo-doctor` also reports a missing peer dependency from an Expo native module, install it directly before rebuilding. Example:

```
npx expo install expo-asset
```

Rule: In checked-in native Expo projects, keep `AppDelegate.swift` aligned with Expo's default template. Do not add access modifiers to `import Expo`.

expo-doctor Non-CNG Warning Is Not a Crash Diagnosis

Symptom: `expo-doctor` fails with:

```
Check for app config fields that may not be synced in a non-CNG project
```

and suggests adding `/ios` to `.gitignore`.

What it actually means: The project has checked-in native `ios/` or `android/` folders, so EAS will NOT automatically sync many `app.json` fields (`orientation`, `icon`, `scheme`, `splash`, `plugins`, etc.) into the native project during build.

Important: This warning does **not** explain a TestFlight startup crash by itself. It is a config-sync warning, not a runtime crash report.

Rule:

- If you intentionally keep native folders checked in, treat this as expected and make native changes explicitly in Xcode / plist / Gradle files when needed.
- Do not chase this warning as the root cause of a crash unless the crash is clearly caused by stale native config.

| expo-audio Can Be Too Aggressive During App Startup

Seen in: [Imperial Dreams](#)

Symptom: App crashes or dies very early in TestFlight, sometimes immediately on launch, with little or no useful crash detail.

Root cause: A root-level audio provider can be too aggressive if it:

- configures the iOS audio session immediately on mount
- creates multiple native `AudioPlayer` instances during app startup
- starts loading music before the app/session has finished hydrating

This may survive local/dev testing but fail on newer iOS/TestFlight builds where startup timing is less forgiving.

Important correction: Early launch timing alone is **not enough** to blame audio. In Imperial Dreams, an instant TestFlight launch crash was first suspected to be audio-related, but the full crash report later showed a native Swift startup trap in `AppDelegate.swift`. If the full `.ips` report shows `EXC_BREAKPOINT / SIGTRAP`, `_assertionFailure`, and frames around `startReactNative(...)`, inspect native startup first.

Safer pattern:

```
const [isAudioBootstrapped, setIsAudioBootstrapped] = useState(false);

useEffect(() => {
  if (!isSessionReady) {
    return;
  }

  const task = InteractionManager.runAfterInteractions(() => {
    setTimeout(() => {
      setAudioModeAsync({
        playsInSilentMode: true,
        shouldPlayInBackground: false,
        allowsRecording: false,
      })
      .then(() => setIsAudioBootstrapped(true))
      .catch(() => {
        // Fail safe: disable audio boot, do not crash app startup
      });
    }, 500);
  });

  return () => task.cancel();
}, [isSessionReady]);
```

Rules:

- Audio should be optional at startup. The app must survive if audio setup fails.
- Defer audio boot until after hydration and initial interactions.
- Lazy-create short UI sound players on first use instead of preallocating them all at launch.

- Do not diagnose an instant launch crash from metadata alone. Get the full crash report before deciding the culprit is audio.

ExpoAppDelegate Must Know About Your Custom React Factory

Seen in: [My Pet Slime](#)

Symptom: TestFlight app crashes almost immediately on launch, often within about `0.1s`, with a crash report that includes:

```
EXC_BREAKPOINT / SIGTRAP
_assertionFailure(_:_:file:line:flags:)
-[RCTReactNativeFactory
startReactNativeWithModuleName:inWindow:initialProperties:launchOptions:]
```

This can happen even though JavaScript bundling succeeds and the app works locally.

In one My Pet Slime release build, the unsymbolicated crash report pointed at a native Swift fatal trap. Disassembling the matching IPA address showed the exact Expo fatal error:

```
recreateRootView: Missing factory in ExpoAppDelegate
```

The usual JS-bundle suspect was not the cause: `main.jsbundle` was present in the IPA. Removed packages such as `expo-widgets` were also not the cause. The app died deterministically on launch because native Expo startup could not recreate the root view.

Root cause: A custom `ios/<AppName>/AppDelegate.swift` created an `ExpoReactNativeFactory`, but only stored it in a local property. It never registered that factory with `ExpoAppDelegate` itself. Expo subscribers may ask the app delegate to recreate the root view during startup; if `ExpoAppDelegate.factory` is still `nil`, native startup can hit a Swift fatal assertion.

Bad pattern:

```
let factory = ExpoReactNativeFactory(delegate: delegate)
reactNativeDelegate = delegate
reactNativeFactory = factory
factory.startReactNative(withModuleName: "main", in: window, launchOptions:
launchOptions)
```

Correct pattern:

```
let factory = ExpoReactNativeFactory(delegate: delegate)
reactNativeDelegate = delegate
reactNativeFactory = factory
bindReactNativeFactory(factory)
factory.startReactNative(withModuleName: "main", in: window, launchOptions:
launchOptions)
```

Rules:

- If you customize Expo's native iOS startup, keep `AppDelegate.swift` aligned with Expo's expected lifecycle, not just React Native's.
 - When creating a custom `ExpoReactNativeFactory`, immediately call `bindReactNativeFactory(factory)`.
 - If a full crash log shows a Swift assertion on the main thread during `startReactNative(...)`, `recreateRootView(...)`, or a fatal trap in `ExpoAppDelegate.swift`, inspect `AppDelegate.swift` before chasing JS causes like audio, gameplay code, missing bundles, or removed packages.
 - A version bump alone does not fix this. The binary must be rebuilt after the `AppDelegate.swift` change, and an already-built IPA still contains the old native startup code.
-

Part 7 — Assets, Dependencies, Code Structure And Routing

The everyday traps that cost hours and have unremarkable fixes.

Asset Issues

| Image Format Mismatch

Observed symptom: an asset with the wrong underlying format caused an iOS asset-processing or validation failure. `npm ci` exited with non-zero code: 1 by itself is only a wrapper message; read the first underlying error before assigning the cause.

Root cause: AI-generated images are often JPEG content saved with `.png` extension.

Diagnose:

```
$ file assets/icon.png
assets/icon.png: JPEG image data # ✗ WRONG!
# Should say: PNG image data
```

Fix: convert to a new output, verify it, then replace intentionally:

```
magick assets/icon.png /tmp/icon-fixed.png
file /tmp/icon-fixed.png
identify -verbose /tmp/icon-fixed.png
```

For an App Store icon, also verify 1024×1024 dimensions and no alpha channel. Avoid an in-place conversion until the output has been inspected.

| iPad Screenshot Requirement

Symptom: "Unable to Add for Review" - requires 13-inch iPad screenshots.

Fix: If app is iPhone-only, disable tablet in `app.json`:

```
"ios": {
  "supportsTablet": false
}
```

Then rebuild and resubmit.

Do this only when the product is genuinely iPhone-only. If the binary still declares iPad support because a checked-in native project has stale settings, update the native target or regenerate it as appropriate and verify the built archive. At the time of this edition Apple requires the 13-inch

set for iPad apps.

Dependency Management

| Legacy Peer Dependencies In These Expo 54 Repos

Several source apps in this folder have peer ranges that conflict under modern npm even though the locked dependency set builds successfully with Expo SDK 54. For those known projects, `legacy-peer-deps` is often the practical compatibility setting:

```
npm install --legacy-peer-deps
```

If a project has demonstrated that requirement, make local and EAS installs consistent with a reviewed project-level `.npmrc`:

```
legacy-peer-deps=true
```

This is not a general npm best practice. npm warns that the option ignores peer dependency contracts. Before enabling it:

1. run `npx expo-doctor` and `npx expo install --check`;
2. inspect the exact `ERESOLVE` chain;
3. align Expo-managed packages with `npx expo install` where possible;
4. retain the lockfile and test the resulting native build; and
5. record why the exception exists and revisit it during an SDK migration.

Do not delete `package-lock.json` as a routine repair. It is the reproducibility record used by `npm ci`. If a clean reinstall is necessary, first try removing only `node_modules` and running `npm ci`; regenerate the lockfile only as a deliberate dependency change with a reviewed diff.

Rule: Expo SDK 54 repos often need `legacy-peer-deps` when their known-good dependency graph has a real peer-range conflict. Use it as a documented exception, not as a way to hide arbitrary dependency drift.

| SDK Version Mismatches

Run `npx expo-doctor` and `npx expo install --check` to identify mismatches. If the proposed changes are understood and reviewed, use:

```
npx expo install --fix
```

Examples observed in this SDK 54 app set—not permanent global version rules:

- `expo-gl` must match SDK version

- `@types/react` for React Native 0.81+ needs `~19.1.10`
-

Code Patterns That Work

| File-Based Routing Structure

Each game folder:

```
src/app/games/mygame/  
├─ _layout.tsx      # Stack navigator  
├─ index.tsx        # Level select  
└─ [levelId].tsx    # Game screen
```

Copy folder, rename, update branch name = new game.

| Relative Import Counting

For `src/app/games/timing/[hourId]/[levelId].tsx`:

- `[levelId].tsx` is a file, not another directory. From its containing `[hourId]/` directory, climb through `[hourId]`, `timing`, `games` and `app`.
- For `src/styles/theme`, use `../../../../styles/theme` from this exact path.

Common mistake: Adding extra `../` because file is IN a folder.

Prefer a configured alias such as `@/styles/theme` for deeply nested routes; it removes this counting problem and survives route reorganisation better.

| Game Screen Pattern

```
export default function GameScreen() {  
  const { levelId } = useLocalSearchParams();  
  const [phase, setPhase] = useState<'ready'|'play'|'result'>('ready');  
  // ... game logic  
}
```

| Constellation Tree Data — App-Specific Registry Pattern

In the puzzle app that used a constellation map, all game layouts lived in `src/data/treeData.ts`:

- Add to `GameBranch` type
 - Add constellation array
 - Add to `BRANCH_COLORS`, `BRANCH_NAMES`, `BRANCH_ICONS`
-

Expo Router Issues

| Diagnose A New Route Instead Of Registering It Blindly

Symptom: You add a new folder such as `src/app/quiz/`, but navigation does not reach it, or a visible control does not respond.

Expo Router discovers eligible route files automatically. Declaring selected `<Stack.Screen>` children is normally for options; it is not a registration list that every route must join. First separate three different failures:

- **Navigation target:** confirm the `href / router.push` string matches the file path, route-group rules and dynamic parameters.
- **Layout warning:** a warning that a named `<Stack.Screen>` has no matching file means the configuration name is stale or misspelled; fix or remove it.
- **Touch delivery:** if the target path is valid but the button never fires, inspect overlays, parent bounds, `pointerEvents`, disabled state and gesture ownership. Adding a screen declaration does not repair hit testing.

Use explicit entries only when that route needs options:

```
// src/app/_layout.tsx
<Stack>
  <Stack.Screen name="index" options={{ headerShown: false }} />
  <Stack.Screen name="quiz" options={{ title: "Quiz" }} />
</Stack>
```

Or configure the navigator once and let the filesystem provide every route:

```
<Stack screenOptions={{ headerShown: false }} />
```

Rule: a missing route is a path/configuration problem; a dead button is often a geometry or gesture problem. Prove which event is failing before changing the navigator.

Part 8 — Warnings: Classify Before Ignoring

Some messages are informational, some predict future migration work, and some identify a real mismatch. Classify them in context rather than suppressing them.

Warning Triage

Message	Immediate meaning	Action
<code>WARN [expo-av]: Expo AV has been deprecated...</code>	The SDK 54 package can still run, but it no longer receives fixes and was removed in SDK 55.	It may be accepted temporarily in a frozen SDK 54 app; create a migration issue and do not use <code>expo-av</code> for new work.
<code>WARN [Layout children]: No route named "games/X" exists...</code>	A named navigator child does not match the discovered route tree.	Fix the stale/misspelled screen name or remove the unnecessary declaration. Do not label it harmless without checking navigation.
<code>> Stopped server after a completed bundle</code>	The local Metro process ended, commonly after Ctrl+C or terminal closure.	Restart it if development should continue; this is not evidence of an app build failure.

Rule: an informational warning can be non-blocking for today's release while still representing real maintenance debt. Record both decisions.

Common Confusing Messages (NOT Errors)

"Stopped server" Message

What you see:

```
iOS Bundled 3067ms node_modules/expo-router/entry.js (1649 modules)
WARN [expo-av]: Expo AV has been deprecated...
> Stopped server
```

What it means: the local Metro process ended, commonly because Ctrl+C was pressed or the terminal closed. This line alone is not an app error.

Solution: Just run the server again:

```
npx expo start --lan --clear
```

How to tell if there's a real error:

- Real errors show stack traces and error messages
 - The bundling should complete (e.g., "1649 modules")
 - Check TypeScript: `npx tsc --noEmit`
-

Part 9 — Building, Submitting And Shipping

Getting from a working local app to a build that App Store Connect will actually accept.

Deploy Workflow

```
# Gate 1 – source and configuration evidence
npm run typecheck
npm test
npx expo-doctor
file assets/*.png
git status --short
git diff --check

# Gate 2 – native validation build when native code changed
eas build --platform ios --profile development

# Gate 3 – release candidate after device validation and sign-off
eas build --platform ios --profile production

# Gate 4 – intentional upload to App Store Connect/TestFlight
eas submit --platform ios

# Gate 5 – App Store Connect (manual review/sign-off)
# - Add the current required 6.9" iPhone screenshots
# - Add 13" iPad screenshots only when the app supports iPad
# - Fill metadata
# - Select the processed build
# - Submit for Review
```

`eas build --auto-submit` combines Gates 3 and 4. Use it only when that upload is already authorised and the submission profile is known-good. It uploads the binary to App Store Connect/TestFlight; it does not fill the product page or press Apple's final “Submit for Review” control.

EAS Submit Config

| Auto-Submit Requires Real App Store IDs

The identifiers below are deliberately fictional examples. Keep real team/app identifiers in the project configuration and do not paste authenticated build logs or credentials into a public version of this guide.

Symptom: `eas build --platform ios --profile production --auto-submit` fails at the very end (after uploading!) with:

Invalid Apple Team ID was specified. It should consist of 10 uppercase letters or digits.
Example: "AB32CZE81F".
Error: build command failed.

Root cause: `eas.json` has placeholder values under `submit.production.ios` (e.g., `YOUR_APP_ID_HERE` / `YOUR_TEAM_ID`). The build succeeds and uploads, but submission fails because the Team ID isn't valid.

How to find your Team ID: Look at the build log output — it's printed during authentication:

```
> Team YOUR_TEAM_NAME (ABCDE12345)    ← This is your Team ID
```

Or check [Apple Developer Account](#) → Membership Details.

Solution: Replace placeholders in `eas.json` with real values:

```
{
  "submit": {
    "production": {
      "ios": {
        "appleTeamId": "ABCDE12345"
      }
    }
  }
}
```

Tip: If you don't need auto-submit, you can remove the entire `submit` block from `eas.json` and submit manually via `eas submit` afterward.

App Store Connect Rejects Duplicate Submissions By Build Number

Symptom: `eas build --platform ios --profile production --auto-submit` uploads successfully, but submission fails with:

You've already submitted this build of the app.
Builds are identified by CFBundleVersion from Info.plist (`expo.ios.buildNumber` in `app.json`).

Root cause: Apple identifies each submitted iOS build by `CFBundleVersion`, which Expo maps from `expo.ios.buildNumber`. If you try to submit another archive with the same build number, App Store Connect rejects it even if the binary was rebuilt.

Solution: bump `expo.ios.buildNumber` in `app.json` before rebuilding:

```
"ios": {
  "bundleIdentifier": "com.example.app",
  "buildNumber": "2"
}
```

Then rebuild and resubmit.

Recommended fix for repeat submissions: enable automatic iOS build-number bumps in

`eas.json`:

```
{
  "build": {
    "production": {
      "autoIncrement": true
    }
  }
}
```

Rule: if you use `--auto-submit`, assume every new App Store submission needs a fresh iOS build number.

| Zustand + Skia: Publishing Mutable Simulation State

Symptom: Either units don't render (passing same array references through Zustand) or the game freezes during touch interactions (shallow-copying large arrays too often).

Root cause: two interacting identity/frequency problems:

1. Zustand selectors and React subscriptions use reference identity. Mutating `gameWorld.units` in place and publishing the same array reference can prevent subscribers from seeing a meaningful change.
2. Shallow copying (`[...gameWorld.units]`) creates a visible snapshot, but doing it from every gesture event and the game loop—more than 100 times per second in this incident—created allocation and garbage-collection pressure.

React Compiler was enabled explicitly in the affected app and can add another memoisation layer, but it is **opt-in in Expo SDK 54** and was not the fundamental reason same-reference mutable state failed to notify subscribers.

```
// ❌ BAD – same mutable reference can be equal to the previous selection
syncRenderState: () => set((state) => ({
  renderFrame: state.renderFrame + 1,
  renderUnits: gameWorld.units,      // same ref = memoized away
})),

// ❌ ALSO BAD – copies work but called 100+/sec from gesture handlers = freeze
export function addPathPoint(x, y) {
  path.push({ x, y });
  syncRenderState(); // creates copies on EVERY gesture event!
}
```

Solution used in this app: publish shallow snapshots from controlled points:

1. `resetGameWorld()` — once at game start (before loop runs)
2. The game loop — measured at 30 fps during gameplay for this workload

Remove `syncRenderState()` from the hot gesture/action handlers (`addPathPoint`, `spawnUnit`, `selectUnitsNear`, `commitPath`, `clearSelection`, and similar paths) when the loop already publishes a bounded snapshot.

```
// ✅ Store – new references make a publishable render snapshot
syncRenderState: () => set((state) => ({
  renderFrame: state.renderFrame + 1,
  renderUnits: [...gameWorld.units],    // new ref each sync
  renderNodes: [...gameWorld.nodes],
})),

// ✅ Gesture handlers – NO sync call, game loop handles it
export function addPathPoint(x: number, y: number) {
  path.push({ x, y });
  // syncRenderState removed – game loop syncs at 30fps
}
```

Result in the measured app/device set: units rendered correctly and 30 snapshot publishes per second stayed within budget without the interaction freeze. Measure array size, allocation rate and frame time in your app; 30 fps is not a universal safe copying rate. For larger worlds, publish compact view models, versioned buffers or UI-thread/native state instead of cloning the whole simulation.

Hot Simulation State Should Not Live In A Root App Context

Symptom: Combat feels frame-by-frame, the app slows down during battles, or iOS kills the app under load even though the actual game rules are simple.

Root cause: A real-time simulation updates frequently (for example every `140ms`), but the live state is stored in a root React context that is also consumed by home screens, campaign screens, audio providers, and other non-combat UI. Every tick republishes a brand-new context value and causes unnecessary rerenders throughout the app.

In strategy games this gets worse when:

- multiple battlefronts simulate at the same time
- AI rebuilds route graphs every tick
- immutable geometry lookups are recalculated repeatedly

Bad pattern:

```
// Root provider updates every tick
setSession((current) => ({
  ...current,
  activeRun: {
    ...current.activeRun,
    sectorStates: recomputeAllSectors(),
  },
}));
```

Better pattern:

```
// 1. Keep stable app/session data in one context
// 2. Put hot combat state in a separate context/store
// 3. Subscribe only theater/sector screens to combat updates
```

```
const SessionContext = createContext(...);
const CombatContext = createContext(...);
```

Also cache immutable data:

```
const adjacencyCache = new WeakMap<SectorMapData, Map<string, string[]>>();
const starMapCache = new WeakMap<SectorMapData, Map<string, StarNodeData>>();
```

Rule: If state changes on a tight loop, do not publish it through the same global context used by the rest of the app.

Appendix A — Public Apps Referenced

The public case studies link to their shipped App Store listings. Internal and unlisted projects are anonymised throughout this edition. Bundle identifiers are deliberately omitted: public bundle IDs are not credentials, but they add little reader value and non-public IDs disclose unnecessary product information.

All published apps: [Eoin Tolster on the App Store](#)

App	Public listing
Localresearch	App Store
QuestGiver	App Store
Three Finger Shoot	App Store
Melody Mind	App Store
DotPhysics	App Store
LongHorizon	App Store
Fly Wheel	App Store
Veranda: Daily Puzzles	App Store
Laneforge	App Store
Junk & Jolt	App Store
My Pet Slime	App Store
Obsidian Wastes	App Store
Imperial Dreams	App Store
RealMath	App Store
Wayline	App Store
Astral And Audio	App Store
Make Army Go Away	App Store

Appendix B — Pre-Flight Checklist

Run the applicable gates before a release build. Keep everyday validation separate from the final external upload.

```
npm run typecheck           # or the repository's equivalent
npm test                    # where the repository defines tests
npx expo-doctor             # SDK and dependency drift
file assets/*.png           # verify content, then dimensions and alpha
git status --short          # review the local working tree
git diff --check            # whitespace/conflict-marker guard
eas build --platform ios --profile production
```

- ☐ `expo-doctor` clean, or every warning understood and deliberate
- ☐ Images are genuinely the format their extension claims
- ☐ `.easignore` / `.gitignore` reviewed; intended archive scope understood
- ☐ Intended source snapshot committed or otherwise recorded for reproducibility
- ☐ Build number incremented past the last submitted one
- ☐ First-run database path tested on a clean install, not a migrated one
- ☐ Release build checked on a real device, not just Expo Go
- ☐ `eas submit` or `--auto-submit` authorised as a separate upload action

Appendix C — Index Of Every Lesson

The lessons below follow document order. This index is navigational rather than a frozen lesson count; headings may be refined as the field manual evolves.

::: {.lesson-index}

Local LLM Setup With Expo And `llama.cpp`

- [What Actually Works](#)
- [Architecture That Worked](#)
- [Model Download And Storage Pattern](#)
- [SQLite Tables That Were Useful](#)
- [Seed And Upsert SQL Must Match Schema Exactly](#)
- `llama.cpp` [Integration Lessons](#)
- [1. Use A Native Expo Module, Not A Hacky JS Wrapper](#)
- [2. CocoaPods File Patterns Must Stay Pod-Local](#)
- [3. `ggml-metal.metal` Needed A Generated Embedded Variant](#)
- [3a. `.incbin` Was Too Fragile For EAS Cloud Builds](#)
- [4. `GGML\_VERSION` And `GGML\_COMMIT` Were Missing](#)
- [5. ARC And Metal Backend Files Did Not Mix By Default](#)
- [6. Swift Import Of Objective-C Errors Can Become `throws`](#)
- [7. Swift 6 Sendability Checks Show Up Fast](#)
- [8. `mtmd\_tokenize` Rejected `bitmaps.data\(\)` Due To C++ Const Strictness](#)
- [9. Expo `file://` URIs Must Be Normalized Before Passing To `PDFDocument`](#)
- [10. Large PDF Render Loops Need `@autoreleasepool` On iOS](#)
- [Recommended Native Runtime Behavior](#)
- [EAS Build Workflow That Made Sense](#)
- [Best Practices For New Apps](#)
- [Quick Checklist For Reuse In A New App](#)

Native Expo Module Development (Finger Shoot)

- [Module Structure \(Critical\)](#)
- [expo-module.config.json](#)
- [Podspec Requirements](#)
- [Camera Permission in Native Code](#)
- [Verifying Autolinking Detection](#)
- [Common Pitfalls](#)

Game Development Patterns (Three Finger Shoot)

- [Preventing Game Loop Crashes \(Multiple State Updates\)](#)
- [Preventing Gesture Spam \(Require State Reset\)](#)
- [Random Values + useMemo \(Preventing Flicker\)](#)
- [Validate App Store Links And Provide A Fallback](#)
- [Non-Intrusive Cross-Promotion Pattern](#)
- [Varied Movement Patterns for Game Objects](#)
- [Native Gesture Detection \(Apple Vision Framework\)](#)
- [Countdown Before Game Start](#)

Large Skia Campaign Case Study — August 2026

- [A Skia Picture Recording Is Live Native Work, Not A Harmless Data Build](#)
- [Expo Router Focus And Component Mounting Are Not The Same Lifecycle](#)
- [Removing A JavaScript Reference Does Not Prove Native Memory Was Freed](#)
- [Loading UI Must Report Real Work](#)
- [Display Refresh Rate Can Silently Double Game Work](#)
- [Read The Analytics Report Type Before Diagnosing A Crash](#)
- [Source Gates, Exports And Device Proof Answer Different Questions](#)
- [Performance Counters Must Follow The Mounted Component Tree](#)
- [A Control Drawn Outside Its Parent May Be Visible But Untappable](#)
- [Transparent Equipment Layers Must Preserve The Base Sprite](#)
- [Floating Sprites Usually Mean The Source Frame And Runtime Anchor Disagree](#)
- [Route Objectives Need A Forward-Path Simulation](#)

Reanimated, Simulation And Layout Case Study — July-August 2026

- [A Reanimated Worklet May Only Call Other Worklets](#)
- [Worklets Capture Identifiers At Module Evaluation, Not At Call Time](#)
- [Reanimated CSS Keyframe Selectors Must Be Percentages Or 0-1 Numbers](#)
- [Never Read A Shared Value During React Render](#)
- [Live Skia Scene Complexity Still Costs Per Frame](#)
- [Anything Derived From A Runtime Dimension Must Be Bounded](#)
- [Clamp The Frame Delta Before Stepping A Simulation](#)
- [React Native Layout Defaults Are Not The Web's, And Layout Bugs Are Arithmetic](#)
- [A Test That Pins Source Text Can Hold A Bug In Place](#)
- [A Glitch At The Same World Position Usually Means A Streaming Boundary](#)
- [The First Plausible Fix Was Real, But Incomplete](#)
- [The Campaign-Wide Cause](#)
- [The Durable Fix](#)
- [Atlas Reuse Must Share Geometry, Not Just The PNG](#)
- [How To Prove This Class Of Fix](#)
- [Short Checklist For Future Skia Game Passes](#)

Critical Production Issues

- [Zustand Persist Hydration Timing](#)
- [An Awaited `expo-av` Operation Stalled A Production Flow](#)
- [A React Native Modal Crashed In These iOS Builds](#)
- [expo-audio Playback Status Can Cause Infinite React Update Loops](#)
- [Expo Swift Import Access-Level Mismatch During EAS Build](#)
- [expo-doctor Non-CNG Warning Is Not a Crash Diagnosis](#)
- [expo-audio Can Be Too Aggressive During App Startup](#)
- [ExpoAppDelegate Must Know About Your Custom React Factory](#)

Asset Issues

- [Image Format Mismatch](#)
- [iPad Screenshot Requirement](#)

Dependency Management

- [Legacy Peer Dependencies In These Expo 54 Repos](#)
- [SDK Version Mismatches](#)

Code Patterns That Work

- [File-Based Routing Structure](#)
- [Relative Import Counting](#)
- [Game Screen Pattern](#)
- [Constellation Tree Data — App-Specific Registry Pattern](#)

Expo Router Issues

- [Diagnose A New Route Instead Of Registering It Blindly](#)

Common Confusing Messages (NOT Errors)

- ["Stopped server" Message](#)

EAS Submit Config

- [Auto-Submit Requires Real App Store IDs](#)
- [App Store Connect Rejects Duplicate Submissions By Build Number](#)
- [Zustand + Skia: Publishing Mutable Simulation State](#)
- [Hot Simulation State Should Not Live In A Root App Context](#)

...

Appendix D — Versioned References

The volatile claims in this manual were checked on **13 August 2026** against the following first-party sources. These links are a release-time checklist, not a substitute for checking the current documentation on the day of a build.

Topic	What This Edition Relies On	First-Party Source
Apple uploads	From 28 April 2026, iOS and iPadOS uploads must be built with the iOS 26 SDK or later.	Apple Developer News
Expo SDK 54 and Xcode	Expo documents SDK 54 EAS images using Xcode 26, which can satisfy that toolchain requirement.	Expo: App Store minimum SDK 26
SDK 54 baseline	SDK 54 maps to React Native 0.81 and React 19.1; its New Architecture behavior and deprecations are version-specific.	Expo SDK 54 changelog
EAS upload scope	<code>.easignore</code> , when present, controls the build upload; otherwise EAS falls back to <code>.gitignore</code> . Inspect the archive rather than assuming uncommitted files are excluded.	Expo <code>.easignore</code> reference
Expo package versions	Prefer <code>npx expo install</code> and <code>npx expo install --fix</code> so packages are selected for the active SDK.	Expo: Using libraries
Peer dependency escape hatch	npm says <code>legacy-peer-deps</code> ignores peer dependency contracts and does not recommend it as a general solution.	npm configuration reference
Native module config	Expo documents <code>apple</code> and granular <code>ios / macos</code> platform keys; CLI resolution examples use <code>--platform apple</code> .	Expo module configuration
File-based routing	Expo Router derives routes from files and uses layout files to configure navigators.	Expo Router layouts
React Compiler	In SDK 54, React Compiler is configured explicitly; do not diagnose every stale mutable reference as compiler memoisation.	Expo React Compiler guide
Store screenshots	Expo's store-assets guide currently lists the 6.9-inch iPhone class and the required iPad class when the app supports iPad.	Expo store assets guide
React Native APIs	<code>Modal</code> and <code>Linking</code> are supported APIs; app-specific failures need logs, lifecycle evidence and fallbacks rather than blanket avoidance.	React Native Modal · React Native Linking

Release rule: run `npx expo-doctor`, verify the selected EAS build image and Xcode version, review the generated upload archive, and re-open the current Apple submission requirements before producing a release candidate.

About The Author

Eoin Tolster is a Forward Deployed AI Engineer based in Australia, working on applied AI systems from prototype through to production — retrieval and document extraction, agent architectures, evaluation, and on-device models. The apps behind this document are personal projects built and shipped alongside that work.

- **YouTube** — youtube.com/@eointolster
- **LinkedIn** — linkedin.com/in/eoin-tolster-2290b6221
- **Website** — theeoin.com

If this saved you an afternoon, the YouTube channel covers the same ground with the code on screen.